ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Đồ án 2 - Khai Thác Tập Phổ Biến

Nghiên cứu, Cài đặt, và Đánh giá

Nguyễn Đỗ Bảo

Trường Đại học Khoa học Tự nhiên

Khoa Công nghệ Thông tin

CSC14004 - Khai thác Dữ liệu và Ứng dụng

Hồ Chí Minh, Tháng 4/2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Đồ án 2 - Khai Thác Tập Phổ Biến

Nghiên cứu, Cài đặt, và Đánh giá

Nguyễn Đỗ Bảo

MSSV: 23127026

GVHD: Lê Nhựt Nam, Th.S
Khoa Công nghệ Thông tin

Đồng GVHD: Nguyễn Ngọc Đức, Th.S
Khoa Công nghệ Thông tin

Nguyễn Thị Thu Hằng, Th.S
Khoa Công nghệ Thông tin

Trường Đại học Khoa học Tự nhiên

Khoa Công nghệ Thông tin

CSC14004 - Khai thác Dữ liệu và Ứng dụng

Hồ Chí Minh, Tháng 4/2026

Mục lục

Danh sách hình vẽ iii

Danh sách bảng 1

1 Nền tảng lý thuyết **2**

1.1 Bài toán khai thác tập phổ biến 2

1.1.1 Cơ sở dữ liệu giao dịch (transaction database) 2

1.1.2 Độ hỗ trợ (support) 2

1.1.3 Tập phổ biến (frequent itemset) 3

1.1.4 Tập đóng (closed itemset) và tập tối đại (maximal itemset) . . . 3

1.1.5 Tính chất Apriori (downward closure) 3

1.2 Phân tích thuật toán AprioriTID 4

1.2.1 Ý tưởng cốt lõi 4

1.2.2 Cấu trúc dữ liệu chính 5

1.2.3 Mã giả (Pseudocode) 5

1.2.4 Phân tích độ phức tạp 7

1.2.5 So sánh lịch sử 9

2 Ví dụ minh họa **11**

2.1 Ví dụ 1: Cơ sở 11

2.2 Ví dụ 2: Tình huống đặc biệt 12

3 Cài đặt **15**

3.1 Môi trường và công cụ 15

3.2 Cấu trúc thư mục 16

3.3 Cài đặt cơ bản 17

3.3.1 Cấu trúc dữ liệu 17

3.3.2 Sinh ứng viên: apriori-gen 17

3.3.3 Vòng lặp chính AprioriTID 19

3.4 Định dạng đầu vào và đầu ra 20

3.4.1 Định dạng SPMF 20

3.4.2 Hàm đọc/ghi 20

3.4.3 Điểm vào CLI 21

3.5 Tối ưu hóa bộ nhớ và tốc độ 21

3.5.1 Ý tưởng: bảng băm tiền tố 21
3.5.2 Đo lường trên 5 bộ dữ liệu toy 22
3.6 Kiểm thử và xác minh 23
3.6.1 Hai file kiểm thử tự động 24
3.6.2 Kết quả 24

4 Thực nghiệm và đánh giá **25**
4.1 Mục tiêu và phương pháp luận 25
4.2 Môi trường và dữ liệu 26
4.2.1 Cấu hình phần cứng và phần mềm 26
4.2.2 Bốn bộ dữ liệu chuẩn 27
4.3 Thực nghiệm (a) — Kiểm tra tính đúng đắn 27
4.4 Thực nghiệm (b) — Thời gian chạy theo *minsup* 29
4.4.1 Kết quả bốn bộ dữ liệu 29
4.4.2 Phân tích hình thái đường cong 29
4.4.3 So sánh với SPMF 31
4.5 Thực nghiệm (c) — Số lượng itemset theo *minsup* 33
4.6 Thực nghiệm (d) — Bộ nhớ đỉnh (basic vs. tối ưu) 35
4.7 Thực nghiệm (e) — Khả năng mở rộng 37
4.8 Thực nghiệm (f) — Ảnh hưởng của độ dài giao dịch 39

5 Ứng dụng thực tế **42**
5.1 Bài toán phân tích giỏ hàng 42
5.2 Bộ dữ liệu Retail 43
5.3 Quy trình và cài đặt 43
5.4 Kết quả phân tích 44
5.4.1 Top luật theo confidence 44
5.4.2 Top luật theo lift 45
5.4.3 Thảo luận kinh doanh 45
5.5 Nhận xét tổng quan 46

Tài liệu tham khảo 47

Danh sách hình vẽ

1.1 Thuật toán AprioriTID tìm tập phổ biến dựa trên sinh ứng viên. 6

2.1 Ví dụ 1 — Các bước thực thi AprioriTID trên CSDL 5 giao dịch với minsup = 2. 11
2.2 Ví dụ 2 — Bước $k = 1$: Tất cả 7 item đều phổ biến. 12
2.3 Ví dụ 2 — Bước $k = 2$: $\overline{C}_2$ có 60 entries, gấp 2.5 lần CSDL gốc (24 entries). . 13

3.1 Kết quả đo biến thể *cơ bản* và *tối ưu* của `apriori_tid` trên 5 bộ dữ liệu toy (minsup tuyệt đối in trực tiếp trong tiêu đề mỗi khối). Script sử dụng bộ đếm `@elapsed` và `@allocated` sau khi làm nóng JIT. 23
3.2 Kết quả chạy kiểm thử. 24

4.1 Runtime theo `minsup` trên bốn bộ dữ liệu chuẩn: cài đặt Julia (biến thể tối ưu) so với SPMF AprioriTID. Trục x đảo chiều (minsup giảm dần từ trái sang phải) để đường cong đi lên phản ánh độ khó tăng. 30
4.2 Số lượng itemset phổ biến theo `minsup` trên bốn bộ dữ liệu. Trục y dạng log để quan sát biên độ tăng trưởng phi tuyến trên các bộ đặc (chess, mushrooms) và tuyến tính nhẹ trên bộ thưa (retail, T10I4D100K). 34
4.3 So sánh cấp phát bộ nhớ giữa biến thể cơ bản và biến thể tối ưu trên bốn bộ dữ liệu chuẩn. Trục y dạng log; khoảng cách giữa hai cột phản ánh hệ số giảm cấp phát ($1.45\times$, $1.65\times$, $8.06\times$, $35.70\times$). 36
4.4 Khả năng mở rộng của AprioriTID trên Retail tại `rel_minsup`= 1.0%: runtime tăng gần như tuyến tính theo $|D|$ ($9.92\times$ runtime cho $10\times$ giao dịch). Đường đứt nét là tham chiếu tuyến tính neo tại điểm $|D| = 8\,816$. 38
4.5 Ảnh hưởng của độ rộng giao dịch w lên runtime và cấp phát bộ nhớ trên dữ liệu tổng hợp ($|D| = 2\,000$, $|I| = 40$, minsup tuyệt đối = 200). Đường cong bẻ gãy rõ rệt giữa $w = 12$ và $w = 20$ khi tỉ số $w/|I|$ đạt 0.5 — thời điểm AprioriTID sinh ra bùng nổ ứng viên do số cặp con tăng theo $\binom{w}{2}$. 40

Danh sách bảng

1.1 Các ký hiệu dùng trong phân tích độ phức tạp. 7
1.2 Độ phức tạp không gian của các cấu trúc chính. 9

3.1 Tổng hợp tăng tốc và giảm cấp phát giữa hai biến thể trên 5 bộ dữ liệu toy. 23
3.2 Năm bộ dữ liệu toy dùng trong `tests/test_correctness.jl`. 24

4.1 Bốn bộ dữ liệu chuẩn dùng cho thực nghiệm (a)–(e). Các thông số được đo trực tiếp trên file đầu vào thay vì trích từ đặc tả gốc, do đó có thể chênh lệch nhẹ so với các báo cáo khác. 27
4.2 Kết quả thực nghiệm (a) — kiểm tra tính đúng đắn so với SPMF. 27
4.3 Thực nghiệm (b) trên Chess ($|D| = 3196$, dày đặc). 29
4.4 Thực nghiệm (b) trên Mushrooms ($|D| = 8416$, dày đặc). 29
4.5 Thực nghiệm (b) trên Retail ($|D| = 88162$, thưa). 29
4.6 Thực nghiệm (b) trên T10I4D100K ($|D| = 100\,000$, thưa, tổng hợp). 30
4.7 So sánh runtime (ms) của cài đặt Julia và SPMF tại cùng `minsup` tuyệt đối. Cột $\rho = t_{\text{Julia}}/t_{\text{SPMF}}$. 31
4.8 Số lượng itemset phổ biến theo `minsup` trên bốn bộ dữ liệu chuẩn (trích từ cột `n_itemsets` của các CSV thực nghiệm (b)). 33
4.9 Thực nghiệm (d) — so sánh cấp phát bộ nhớ và runtime giữa biến thể cơ bản và biến thể tối ưu tại một `minsup` trung bình cho mỗi dataset. Tỉ lệ > 1 nghĩa là tối ưu giảm/nhanh hơn bấy nhiêu lần. 35
4.10 Thực nghiệm (e) — khả năng mở rộng trên Retail với `rel_minsup`= 1.0%. . 38
4.11 Thực nghiệm (f) — runtime và cấp phát theo độ rộng giao dịch w trên dữ liệu tổng hợp ($|D| = 2000$, $|I| = 40$, `minsup`=200). 39

5.1 Thống kê cơ bản của bộ dữ liệu Retail. 43
5.2 Top 10 luật theo *confidence* tại *minsup* $= 500$, *minconf* $= 0.30$. 44
5.3 Top 10 luật theo *lift*. Các cặp song hướng ($A \Rightarrow B$ và $B \Rightarrow A$) được giữ nguyên để thấy sự bất đối xứng về confidence. 45

1

Nền tảng lý thuyết

Khai thác tập phổ biến (*frequent itemset mining*) là bài toán nền tảng của lĩnh vực khai phá luật kết hợp, được khởi xướng bởi Agrawal, Imieliński và Swami (Agrawal et al., 1993) và sau đó được hệ thống hoá bởi Agrawal và Srikant qua họ thuật toán Apriori/AprioriTID (Agrawal et al., 1994). Chương này tóm tắt các định nghĩa chính thức được dùng xuyên suốt báo cáo và mô tả chi tiết thuật toán AprioriTID theo trình bày gốc; các hệ quả về độ phức tạp và các cấu trúc dữ liệu có liên quan đều dựa trên phân tích kinh điển trong Han, Kamber và Pei (Han et al., 2011).

1.1 Bài toán khai thác tập phổ biến

1.1.1 Cơ sở dữ liệu giao dịch (transaction database)

Cơ sở dữ liệu giao dịch (CSDL) $\mathcal{D}$ bao gồm một tập hợp các giao dịch T, trong đó mỗi giao dịch chứa một tập hợp các hạng mục không rỗng (*itemset*) $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ sao cho $T \subseteq \mathcal{I}$ và được gán với một định danh giao dịch duy nhất (TID). Ta nói giao dịch T chứa X (với X là tập các hạng mục trong $\mathcal{I}$), nếu $X \subseteq T$.

1.1.2 Độ hỗ trợ (support)

Độ hỗ trợ (*support*) của một tập hạng mục X là phần trăm số lượng giao dịch có chứa X trong cơ sở dữ liệu giao dịch $\mathcal{D}$, có thể được định nghĩa bằng (1.1):

$$sup(X) = \frac{|\{T \mid X \subseteq T \wedge T \in \mathcal{D}\}|}{|\mathcal{D}|} \tag{1.1}$$

Có hai cách để biểu diễn độ hỗ trợ là **độ hỗ trợ tuyệt đối** (*absolute support* hay *support count*) và **độ hỗ trợ tương đối** (*relative support*).

Độ hỗ trợ tuyệt đối của một tập hạng mục X là số lượng giao dịch trong CSDL chứa

X, được định nghĩa bằng:

$$support_count(X) = |\{T \mid X \subseteq T \wedge T \in \mathcal{D}\}| \tag{1.2}$$

Độ hỗ trợ tương đối của một tập hạng mục X là tỷ lệ giữa độ hỗ trợ tuyệt đối của X và tổng số giao dịch trong CSDL, biểu diễn bằng (1.1).

1.1.3 Tập phổ biến (frequent itemset)

Ta nói X là **tập phổ biến** khi $sup(X) \geq minsup$ trong đó $minsup$ là ngưỡng hỗ trợ tối thiểu được định trước. Một tập hạng mục chứa k hạng mục là một **k-itemset**. Tập hợp của các tập phổ biến k-itemset được ký hiệu là L_k.

1.1.4 Tập đóng (closed itemset) và tập tối đại (maximal itemset)

Tập hạng mục X được gọi là **đóng** (*closed*) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp (*superitemset*) Y (tức là, mọi phần tử của X đều thuộc Y, nhưng tồn tại ít nhất một phần tử của Y không thuộc X) nào chứa X mà Y có cùng số hỗ trợ (*support count*) với X trong D, hay nói cách khác:

$$\nexists Y \mid (X \subset Y) \wedge (sup(X) = sup(Y)) \tag{1.3}$$

Tập hạng mục X là **tập phổ biến đóng** (*closed frequent itemset*) khi thỏa mãn cả tính đóng và tính phổ biến.

Tập hạng mục X được gọi là **tập tối đại** (*maximal itemset*) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp Y nào chứa X trong D, hay nói cách khác:

$$\nexists Y \mid (X \subset Y) \tag{1.4}$$

Tập hạng mục X là **tập phổ biến tối đại** (*maximal frequent itemset*) khi thỏa mãn cả tính tối đại và tính phổ biến.

Mối quan hệ giữa tập phổ biến (FI), tập phổ biến đóng (CFI), và tập phổ biến tối đại (MFI) được biểu diễn như sau: MFI $\subseteq$ CFI $\subseteq$ FI.

1.1.5 Tính chất Apriori (downward closure)

Tính chất Apriori: Tất cả các tập con không rỗng của một tập hạng mục phổ biến cũng phải là các tập phổ biến.

Tính chất đơn điệu giảm (antimonotonicity): Nếu một tập hợp không vượt qua được một phép thử, thì tất cả các tập hợp cha của nó cũng sẽ không vượt qua được phép thử đó.

Chứng minh: Nếu một tập hạng mục I không thỏa mãn điều kiện về độ hỗ trợ tối thiểu

($minsup$), thì I sẽ không phải là tập phổ biến (tức là $P(I) < minsup$). Khi đó, nếu có một hạng mục a được thêm vào I, thì tập hạng mục $(I \cup \{a\})$ sẽ không thể xuất hiện nhiều lần hơn I. Vả lại, có thể coi $(I \cup \{a\})$ là tập cha của I (vì $I \subset (I \cup \{a\})$). Do đó, $(I \cup \{a\})$ cũng không thể là tập phổ biến (do $P(I \cup \{a\}) < minsup$).

1.2 Phân tích thuật toán AprioriTID

Phần này dựa trực tiếp trên trình bày gốc của Agrawal và Srikant (Agrawal et al., 1994). Thuật toán tìm tập hạng mục lớn (*large itemsets*) hoạt động lặp đi lặp lại qua nhiều lượt quét CSDL. Ở lượt đầu tiên, thuật toán đếm độ hỗ trợ (*support*) của từng hạng mục để tìm các large 1-itemset. Ở các vòng tiếp theo, các large itemset từ vòng trước được dùng làm cơ sở sinh ra các tập ứng viên (*candidate itemsets*). Thuật toán tiếp tục quét CSDL để đếm support cho ứng viên, lọc ra các large itemset mới và lặp lại quá trình này cho đến khi không tìm thêm được tập nào nữa.

1.2.1 Ý tưởng cốt lõi

Thuật toán Apriori tìm tập phổ biến thông qua quá trình sinh ứng viên và kiểm tra theo từng mức độ dài, trong đó các large k-itemset được sử dụng để khám phá các large (k+1)-itemset. Ý tưởng cốt lõi của AprioriTID dựa trên một biến thể tối ưu hóa gọi là giảm thiểu giao dịch (*transaction reduction*). Nếu một giao dịch không chứa bất kỳ k-itemset phổ biến nào thì nó cũng không thể chứa bất kỳ (k+1)-itemset phổ biến nào khác. Do đó, ta loại bỏ các TID này và không cần xét đến chúng trong các lần quét CSDL ở các vòng lặp tiếp theo. Thay vì phải quét lại CSDL gốc nhiều lần, AprioriTID tạo ra một tập dữ liệu mã hóa trung gian, ký hiệu là $\overline{C}_k$ để lưu vết những ứng viên có mặt trong từng giao dịch. Ở các vòng lặp sau, tập $\overline{C}_k$ này có thể trở nên nhỏ hơn rất nhiều so với kích thước của CSDL gốc, giúp tiết kiệm đáng kể chi phí đọc đĩa (I/O).

Đầu tiên, tìm tập hợp 1-itemset phổ biến bằng cách đếm số lần xuất hiện của TID chứa 1-itemset, và thu thập những 1-itemset thỏa mãn minsup. Tập hợp kết quả được ký hiệu là L_1. Tiếp theo, L_1 được sử dụng để tìm L_2, tập hợp các 2-itemset phổ biến, được sử dụng để tìm L_3, và cứ thế tiếp tục cho đến khi không còn k-itemset phổ biến nào được tìm thấy nữa.

Các thuật toán đi trước như AIS (Agrawal et al., 1993) hay SETM (Houtsma et al., 1995) sinh các tập ứng viên ngay tức thì (on-the-fly) bằng cách đọc từng giao dịch và mở rộng các tập phổ biến có trong giao dịch đó. Cách này tạo ra vô số ứng viên dư thừa, dẫn đến lãng phí tài nguyên. AprioriTID khắc phục điều này bằng cách sinh ứng viên một cách độc lập thông qua việc nối các tập phổ biến bước trước, tạo ra tập ứng viên nhỏ hơn rất nhiều.

1.2.2 Cấu trúc dữ liệu chính

AprioriTID sử dụng một cấu trúc dữ liệu mảng tuần tự đặc trưng là tập $\overline{C}_k$. Mỗi phần tử trong $\overline{C}_k$ đại diện cho một giao dịch, có định dạng: $\langle \text{TID}, \{X_k\}\rangle$, trong đó X_k là k-itemset xuất hiện trong TID. Nếu một giao dịch không chứa bất kỳ ứng viên nào, nó sẽ không có mục (*entry*) tương ứng trong $\overline{C}_k$.

Lấy ví dụ minh họa CSDL $\mathcal{D}$ ở Hình 2.1 với TID 101 chứa các mục: $\{1, 2, 3, 4\}$. Ngưỡng $minsup = 2$.

- Ở bước $k = 1$, $\overline{C}_1$ sẽ là: $\langle$ 101, $\{\{1\}, \{2\}, \{3\}, \{4\}\}$ $\rangle$.
- Ở bước $k = 2$, tập ứng viên C_2 chứa $\{1, 2\}$, $\{1, 4\}$, $\{1, 5\}$,... (lưu ý mục $\{3\}$ không đạt minsup ở L_1 nên các tập chứa $\{3\}$ không xuất hiện trong C_2). Vì TID 101 chứa các tổ hợp con hợp lệ nằm trong C_2 là $\{1, 2\}$, $\{1, 4\}$, $\{2, 4\}$, nên $\overline{C}_2$ cho TID 101 sẽ là: $\langle$ 101, $\{\{1, 2\}, \{1, 4\}, \{2, 4\}\}$ $\rangle$.
- Ở bước $k = 3$, tập ứng viên C_3 là $\{\{1, 2, 4\}, \{1, 4, 5\}\}$. Giao dịch 101 chỉ chứa tổ hợp $\{\{1, 2, 4\}\}$ có mặt trong C_3, nên $\overline{C}_3$ cho TID 101 sẽ là: $\langle$ 101, $\{\{1, 2, 4\}\}$ $\rangle$.
- Ở bước $k = 4$, vì $L_3 = \{\{1, 2, 4\}, \{1, 4, 5\}\}$ không có hai phần tử nào trùng tiền tố độ dài 2, nên hàm apriori-gen trả về $C_4 = \emptyset$ và vòng lặp kết thúc.

1.2.3 Mã giả (Pseudocode)

Giải thích mã giả:

1. (1) Tìm tập hợp các tập phổ biến có kích thước bằng 1 (L_1);
2. (2) Khởi tạo $\overline{C}_1 = \mathcal{D}$. Mỗi item trong mỗi giao dịch được xem là một tập 1-itemset, tạo thành định dạng $\langle$TID, tập các 1-itemset$\rangle$;
3. (3) Bắt đầu vòng lặp tìm các tập phổ biến kích thước k ($k \geq 2$). Vòng lặp này sẽ tiếp tục cho đến khi $L_{k-1} = \emptyset$;
4. (4) Sinh ra tập các ứng viên kích thước k (C_k) từ tập phổ biến L_{k-1}. Hàm apriori-gen dùng để sinh ra tập các ứng viên k-itemset C_k từ tập phổ biến L_{k-1};
5. (5) Khởi tạo tập $\overline{C}_k$ rỗng;
6. (6 - 11) Duyệt qua $\overline{C}_{k-1}$ để xác định và tăng biến đếm độ hỗ trợ cho các ứng viên trong C_k thực sự xuất hiện trong mỗi giao dịch. Sau đó xây dựng $\overline{C}_k$, chỉ giữ lại những giao dịch có chứa ít nhất một ứng viên k-itemset;
7. (12) Lọc ra tập phổ biến L_k. Chỉ giữ lại những ứng viên trong C_k có count $\geq$ *minsup*;
8. (14) Hợp nhất toàn bộ các tập L_k là danh sách tất cả các itemset phổ biến trong CSDL.

Hàm `apriori-gen` nhận đầu vào là L_{k-1}, tức là tập hợp tất cả các tập large $(k-1)$-itemset, và trả về một tập hợp gồm các large k-itemset. Nó gồm hai bước: (1) **join** — hợp L_{k-1} với chính nó để sinh ra các tập ứng viên C_k; và (2) **prune** — tỉa tất cả các ứng viên $c \in C_k$ nếu tồn tại bất kỳ $(k-1)$-itemset nào của c không nằm trong L_{k-1} (xem thủ tục `has_infrequent_subset` trong Hình 1.1).

Thuật toán AprioriTID
Input:

- $\mathcal{D}$: cơ sở dữ liệu giao dịch;
- *minsup*: ngưỡng độ hỗ trợ tối thiểu.

Output: L, tập các itemset phổ biến trong $\mathcal{D}$.
Method:

(1) $L_1 \leftarrow$ {các 1-itemset phổ biến}
(2) $\overline{C}_1 \leftarrow \mathcal{D}$
(3) **for** $k \leftarrow 2$; $L_{k-1} \neq \emptyset$; k++ **do**
(4) $\quad C_k \leftarrow$ apriori-gen(L_{k-1}) ▹ sinh ứng viên mới
(5) $\quad \overline{C}_k \leftarrow \emptyset$
(6) $\quad$ **for all** phần tử $t \in \overline{C}_{k-1}$ **do**
(7) $\quad\quad C_t \leftarrow \{c \in C_k \mid (c-c[k]) \in t.\text{set-of-itemsets} \wedge (c-c[k-1]) \in t.\text{set-of-itemsets}\}$
(8) $\quad\quad$ **for all** ứng viên $c \in C_t$ **do**
(9) $\quad\quad\quad c.\text{count} \leftarrow c.\text{count} + 1$
(10) $\quad\quad$ **end for**
(11) $\quad\quad$ **if** $C_t \neq \emptyset$ **then**
(12) $\quad\quad\quad \overline{C}_k \leftarrow \overline{C}_k \cup \{\langle t.\text{TID}, C_t\rangle\}$
(13) $\quad\quad$ **end if**
(14) $\quad$ **end for**
(15) $\quad L_k \leftarrow \{c \in C_k \mid c.\text{count} \geq \mathit{minsup}\}$
(16) **end for**
(17) **return** $L = \bigcup_k L_k$

procedure apriori-gen(L_{k-1}: tập phổ biến $(k-1)$-itemset)**:**

(1) **for all** itemset $l_1 \in L_{k-1}$ **do**
(2) $\quad$ **for all** itemset $l_2 \in L_{k-1}$ **do**
(3) $\quad\quad$ **if** $(l_1[1] = l_2[1]) \wedge \ldots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$ **then**
(4) $\quad\quad\quad c \leftarrow l_1 \bowtie l_2$ ▹ bước join: sinh ứng viên
(5) $\quad\quad\quad$ **if** has_infrequent_subset(c, L_{k-1}) **then**
(6) $\quad\quad\quad\quad$ **delete** c ▹ bước prune: loại ứng viên không đạt
(7) $\quad\quad\quad$ **else**
(8) $\quad\quad\quad\quad$ thêm c vào C_k
(9) $\quad\quad\quad$ **end if**
(10) $\quad\quad$ **end if**
(11) $\quad$ **end for**
(12) **end for**
(13) **return** C_k

procedure has_infrequent_subset(c: ứng viên k-itemset; L_{k-1}: tập phổ biến $(k-1)$-itemset)**:**

(1) **for all** $(k-1)$-subset s của c **do**
(2) $\quad$ **if** $s \notin L_{k-1}$ **then**
(3) $\quad\quad$ **return** TRUE
(4) $\quad$ **end if**
(5) **end for**
(6) **return** FALSE

Hình 1.1: *Thuật toán AprioriTID tìm tập phổ biến dựa trên sinh ứng viên.*

Lấy ví dụ CSDL $\mathcal{D}$ ở Hình 2.1 có L_2: {{1 2}, {1 4}, {1 5}, {2 4}, {4 5}}. Bước join ghép các cặp có cùng item đầu tiên và item thứ hai theo thứ tự tăng dần: {1 2}⋈{1 4}, {1 2}⋈{1 5}, {1 4}⋈{1 5}, sinh ra C_3: {{1 2 4}, {1 2 5}, {1 4 5}}. Bước prune xóa {1 2 5} vì tập con {2 5} không nằm trong L_2. Do đó chỉ còn lại {{1 2 4}, {1 4 5}} ở C_3.

1.2.4 Phân tích độ phức tạp

Ký hiệu	Ý nghĩa
n	Số giao dịch trong CSDL $\mathcal{D}$
$\|I\|$	Số lượng hạng mục (item) phân biệt
w	Độ rộng trung bình của một giao dịch
$k_{\max}$	Độ dài tập phổ biến dài nhất
$\|C_k\|$	Số ứng viên ở bước k
$\|L_k\|$	Số tập phổ biến ở bước k
$\|\overline{C}_k\|$	Tổng số cặp ⟨TID, itemset⟩ trong $\overline{C}_k$
minsup	Ngưỡng độ hỗ trợ tối thiểu

Bảng 1.1: *Các ký hiệu dùng trong phân tích độ phức tạp.*

Phân rã chi phí theo từng bước

Bước 1 — Khởi tạo. Thuật toán quét toàn bộ CSDL $\mathcal{D}$ một lần duy nhất để tính độ hỗ trợ từng 1-itemset và khởi tạo $\overline{C}_1$. Độ phức tạp:

$$T_{\text{init}} = O(n \cdot w) \tag{1.5}$$

Bước 2 — Sinh ứng viên bằng `apriori-gen`. Hàm `apriori-gen` gồm hai pha con:

- *Join:* duyệt mọi cặp $(p, q) \in L_{k-1} \times L_{k-1}$, mỗi phép so sánh tốn $O(k)$ để kiểm tra $k-2$ item đầu khớp và item cuối theo thứ tự:

$$T_{\text{join}}(k) = O\big(|L_{k-1}|^2 \cdot k\big) \tag{1.6}$$

- *Prune:* với mỗi ứng viên $c \in C_k$, sinh k tập con kích thước $(k-1)$ và kiểm tra thành viên trong L_{k-1}:

$$T_{\text{prune}}(k) = O(|C_k| \cdot k \cdot |L_{k-1}|) \tag{1.7}$$

Tổng chi phí `apriori-gen` tại bước k:

$$T_{\text{gen}}(k) = O\big(|L_{k-1}|^2 \cdot k \ + \ |C_k| \cdot k \cdot |L_{k-1}|\big) \tag{1.8}$$

Bước 3 — Đếm support qua $\overline{C}_k$. Đây là điểm then chốt phân biệt AprioriTID với Apriori cơ bản: thay vì quét lại $\mathcal{D}$, thuật toán duyệt cấu trúc trung gian $\overline{C}_{k-1}$. Với mỗi phần tử $t \in \overline{C}_{k-1}$, thuật toán xác định tập ứng viên C_t có trong t, tăng c.count cho mọi

$c \in C_t$, rồi ghi $\langle t.\text{TID}, C_t \rangle$ vào $\overline{C}_k$ nếu $C_t \neq \emptyset$. Chi phí:

$$T_{\text{count}}(k) = O\left(|\overline{C}_k| \cdot k\right) \tag{1.9}$$

Khi k tăng, $|\overline{C}_k|$ thường giảm nhanh do transaction reduction, nhỏ hơn rất nhiều so với chi phí $O(n \cdot |C_k|)$ nếu phải quét lại $\mathcal{D}$.

Tổng chi phí thời gian. Gộp cả ba bước:

$$T_{\text{total}} = O(n \cdot w) + \sum_{k=2}^{k_{\max}} \left[|L_{k-1}|^2 \cdot k \ + \ |C_k| \cdot k \cdot |L_{k-1}| \ + \ |\overline{C}_k| \cdot k \right]. \tag{1.10}$$

Độ phức tạp thời gian theo trường hợp

Trường hợp tốt nhất. Xảy ra khi dữ liệu thưa và *minsup* cao: $|L_k|$ nhỏ, $k_{\max}$ nhỏ, và $|\overline{C}_k|$ giảm nhanh sau $k = 2$. Khi đó chi phí $T_{\text{count}}(k)$ chiếm ưu thế nhưng $\sum_k |\overline{C}_k| \ll n \cdot k_{\max}$, và thuật toán dừng sớm sau vài vòng lặp. Chi phí xấp xỉ:

$$T_{\text{best}} = O(n \cdot w + |L_1|^2 + |\overline{C}_2| \cdot 2) \tag{1.11}$$

AprioriTID vượt trội so với Apriori chuẩn nhờ tránh được $k_{\max}$ lần quét lại $\mathcal{D}$.

Trường hợp trung bình. Trên CSDL giao dịch thực tế điển hình, số tập phổ biến có phân bố thưa dần theo k: $|L_k|$ đạt cực đại quanh $k = 2$ hoặc $k = 3$ rồi giảm dần, và $|\overline{C}_k|$ co lại nhanh ở các bước sau. Chi phí bị chi phối bởi vài vòng lặp đầu tiên:

$$T_{\text{avg}} = O\left(n \cdot w + \sum_{k=2}^{k_{\max}} |\overline{C}_k| \cdot k \right) \tag{1.12}$$

với tổng $\sum_k |\overline{C}_k|$ thường chỉ là bội số hằng số của $n \cdot w$. Đây là chế độ hoạt động mà AprioriTID được thiết kế để tối ưu.

Trường hợp xấu nhất. Xảy ra khi dữ liệu **dày đặc** và *minsup* thấp: gần như mọi itemset đều phổ biến, $|L_k|$ tăng theo hướng tổ hợp $\binom{|I|}{k}$, và $|\overline{C}_k|$ không co lại. Khi đó số ứng viên tối đa là $O(2^{|I|})$ và độ phức tạp thời gian trở thành *hàm mũ* theo $|I|$:

$$T_{\text{worst}} = O\left(2^{|I|} \cdot |I|\right). \tag{1.13}$$

Đây là giới hạn lý thuyết chung cho mọi thuật toán sinh-và-kiểm-tra (*generate-and-test*) dựa trên Apriori.

Cấu trúc	Không gian	Ghi chú
CSDL gốc $\mathcal{D}$	$O(n \cdot w)$	Lưu trữ đầu vào
C_k, L_k mỗi vòng	$O(\lvert C_k \rvert \cdot k)$	Tập ứng viên và tập phổ biến
$\overline{C}_k$ (đặc trưng AprioriTID)	$O(n \cdot \lvert C_k \rvert)$	Lớn nhất ở $k = 2$, giảm dần

Bảng 1.2: *Độ phức tạp không gian của các cấu trúc chính.*

Độ phức tạp không gian

Điểm đáng chú ý là $\overline{C}_k$ ở giai đoạn $k = 2$ có thể **lớn hơn** $\mathcal{D}$ gốc: mỗi giao dịch w item ánh xạ thành $\binom{w}{2}$ cặp ứng viên, dẫn tới tăng *bậc hai* theo w. Đây là điểm yếu bộ nhớ chính của AprioriTID so với Apriori tiêu chuẩn, đặc biệt rõ trên dữ liệu dày đặc (xem minh họa ở Chương 2).

Các yếu tố ảnh hưởng đến hiệu năng

Ba yếu tố chính quyết định hiệu năng thực tế của AprioriTID:

- **Ngưỡng *minsup*.** Yếu tố ảnh hưởng mạnh nhất. *minsup* thấp làm $|L_k|$, $|C_k|$ và $|\overline{C}_k|$ tăng nhanh theo cấp số, kéo theo cả chi phí thời gian lẫn bộ nhớ. Ngược lại, *minsup* cao giúp transaction reduction cắt tỉa $\overline{C}_k$ nhanh chóng, đưa thuật toán về gần trường hợp tốt nhất.
- **Kích thước và mật độ của CSDL.** Số giao dịch n chỉ ảnh hưởng tuyến tính ở bước khởi tạo $O(n \cdot w)$. Ngược lại, *mật độ* của dữ liệu (tỷ lệ item mỗi giao dịch chứa) mới là yếu tố then chốt: dữ liệu dày đặc (chess, mushroom) làm $|\overline{C}_2|$ phình to và $k_{\max}$ dài hơn, còn dữ liệu thưa (retail, T10I4D100K) giúp thuật toán hội tụ nhanh.
- **Độ rộng trung bình giao dịch w.** Ảnh hưởng *bậc hai* lên kích thước $\overline{C}_2$ (mỗi giao dịch sinh $\binom{w}{2}$ cặp ứng viên) và ảnh hưởng tuyến tính lên bước khởi tạo. Giao dịch càng dài thì cả thời gian lẫn bộ nhớ cao điểm đều tăng.

Ba yếu tố này sẽ được kiểm chứng bằng thực nghiệm trong Chương 4.

1.2.5 So sánh lịch sử

Trong giai đoạn đầu của lĩnh vực khai phá tập phổ biến (FIM), các thuật toán tiên phong như AIS (Agrawal et al., 1993) và SETM (Houtsma et al., 1995) gặp nhiều hạn chế do phải sinh ứng viên "on-the-fly"(trên đường bay) trong quá trình đọc giao dịch và đòi hỏi việc rà quét, sắp xếp cơ sở dữ liệu gốc liên tục rất tốn kém. Sự ra đời của thuật toán Apriori và AprioriTID vào năm 1994 bởi R. Agrawal và R. Srikant (Agrawal et al., 1994) đã tạo ra một bước ngoặt lớn khi giải quyết được những nhược điểm này. Điểm cải tiến mang tính lịch sử của AprioriTID là nó áp dụng hàm sinh ứng viên tiên nghiệm giúp thu hẹp không gian tìm kiếm, đồng thời sử dụng cấu trúc dữ liệu trung gian mã hóa ($\overline{C}_k$) để đếm độ hỗ trợ. Nhờ vậy, thuật toán không cần phải quét lại cơ sở dữ liệu gốc từ vòng lặp thứ hai trở đi, giúp giảm thiểu đáng kể chi phí truy xuất đĩa (I/O) so với các thuật toán tiền nhiệm.

Mặc dù là một bước tiến lớn, AprioriTID vẫn để lại một hạn chế cốt lõi: nó vẫn hoàn toàn phụ thuộc vào mô hình "sinh và kiểm tra"(generate-and-test), sinh ra vô số ứng viên nhỏ không cần thiết và dễ gây quá tải bộ nhớ khi xử lý các tập dữ liệu lớn. Để giải quyết bài toán này, các thế hệ thuật toán FIM sau đó đã thay đổi hoàn toàn cách tiếp cận. Nổi bật là thuật toán FP-Growth (Han et al., 2000) ra đời bằng cách áp dụng cấu trúc cây nén FP-tree kết hợp với chiến lược "chia để trị", giúp khai phá trực tiếp các tập phổ biến mà hoàn toàn không cần phải sinh ứng viên. Song song đó, thuật toán Eclat (Zaki, 2000) cũng xuất hiện, giải quyết điểm nghẽn bằng cách chuyển sang sử dụng định dạng dữ liệu dọc (vertical data format), cho phép tính toán độ hỗ trợ cực kỳ nhanh chóng và hiệu quả thông qua phép giao các tập mã giao dịch (TID_set).

2

Ví dụ minh họa

2.1 Ví dụ 1: Cơ sở

CSDL

TID	Items
101	1 2 3 4
102	1 2
103	1 4 5
104	1 2 4 5
105	5

$\overline{C}_1$

TID	Tập Itemset
101	{{1}, {2}, {3}, {4}}
102	{{1}, {2}}
103	{{1}, {4}, {5}}
104	{{1}, {2}, {4}, {5}}
105	{5}

L_1

Itemset	Support
{1}	4
{2}	3
{4}	3
{5}	3

C_2

Itemset
{1 2}
{1 4}
{1 5}
{2 4}
{2 5}
{4 5}

$\overline{C}_2$

TID	Tập Itemset
101	{{1 2}, {1 4}, {2 4}}
102	{{1 2}}
103	{{1 4}, {1 5}, {4 5}}
104	{{1 2}, {1 4}, {1 5}, {2 4}, {2 5}, {4 5}}

L_2

Itemset	Support
{1 2}	3
{1 4}	3
{1 5}	2
{2 4}	2
{4 5}	2

C_3

Itemset
{1 2 4}
{1 2 5}
{1 4 5}
{2 4 5}

$\overline{C}_3$

TID	Tập Itemset
101	{{1 2 4}}
103	{{1 4 5}}
104	{{1 2 4}, {1 2 5}, {1 4 5}, {2 4 5}}

L_3

Itemset	Support
{1 2 4}	2
{1 4 5}	2

Hình 2.1: *Ví dụ 1 — Các bước thực thi AprioriTID trên CSDL 5 giao dịch với minsup = 2.*

Dừng tại $k = 4$**:** Áp dụng *apriori-gen* trên $L_3 = \{\{1, 2, 4\}, \{1, 4, 5\}\}$, bước *join* yêu cầu hai itemset phải trùng $k - 2 = 2$ item đầu. Ở đây $\{1, 2, 4\}[1{:}2] = \{1, 2\}$ khác $\{1, 4, 5\}[1{:}2] = \{1, 4\}$, nên không sinh được ứng viên nào, tức $C_4 = \emptyset$. Thuật toán kết thúc.

Tập kết quả cuối cùng (với support tương ứng):

Itemset	Support	Itemset	Support
{1}	4	{1,5}	2
{2}	3	{2,4}	2
{4}	3	{4,5}	2
{5}	3	{1,2,4}	2
{1,2}	3	{1,4,5}	2
{1,4}	3		

Tổng cộng thu được **11 tập phổ biến**.

Kiểm tra chéo (cross-check) bằng liệt kê thủ công. Duyệt toàn bộ CSDL gốc:

- **1-itemset:** {1} xuất hiện trong T101, T102, T103, T104 ⇒ sup=4. {2} trong T101, T102, T104 ⇒ sup=3. {3} chỉ trong T101 ⇒ sup=1 (*loại*). {4} trong T101, T103, T104 ⇒ sup=3. {5} trong T103, T104, T105 ⇒ sup=3.
- **2-itemset phổ biến:** {1,2} (T101,T102,T104)=3; {1,4} (T101,T103,T104)=3; {1,5} (T103,T104)=2; {2,4} (T101,T104)=2; {4,5} (T103,T104)=2. Cặp {2,5} chỉ xuất hiện trong T104 nên bị loại.
- **3-itemset phổ biến:** {1,2,4} (T101,T104)=2; {1,4,5} (T103,T104)=2. Các tổ hợp khác hoặc chứa item không phổ biến, hoặc có support < 2.
- **4-itemset:** {1,2,4,5} chỉ xuất hiện trong T104 ⇒ sup=1 (*loại*).

Kết quả liệt kê thủ công đúng **11 tập phổ biến**, khớp hoàn toàn với đầu ra của thuật toán ở Hình 2.1.

2.2 Ví dụ 2: Tình huống đặc biệt

Điểm yếu nổi tiếng nhất của AprioriTID là ở giai đoạn $k = 2$, tập $\overline{C}_2$ có thể **lớn hơn rất nhiều** so với CSDL gốc $\mathcal{D}$. Điều này xảy ra khi dữ liệu dày đặc (dense): mỗi giao dịch chứa nhiều item, sinh ra số lượng cặp ứng viên rất lớn.

Ta thiết kế CSDL $\mathcal{D}'$ gồm 4 giao dịch, mỗi giao dịch chứa 6 item từ tập 7 item, với minsup = 2:

Database $\mathcal{D}'$

TID	Items
101	1 2 3 4 5 6
102	1 2 3 4 5 7
103	1 2 3 4 6 7
104	1 2 3 5 6 7

$\overline{C}_1$

TID	Tập Itemset
101	{{1},{2},{3},{4},{5},{6}}
102	{{1},{2},{3},{4},{5},{7}}
103	{{1},{2},{3},{4},{6},{7}}
104	{{1},{2},{3},{5},{6},{7}}

L_1

Itemset	Support
{1}	4
{2}	4
{3}	4
{4}	3
{5}	3
{6}	3
{7}	3

Hình 2.2: *Ví dụ 2 — Bước $k = 1$: Tất cả 7 item đều phổ biến.*

Phân tích kích thước $\overline{C}_1$: CSDL gốc $\mathcal{D}'$ có $4 \times 6 = 24$ mục (item entries). Tập $\overline{C}_1$ cũng

chứa đúng $4 \times 6 = 24$ mục (1-itemset entries) vì tất cả item đều phổ biến, không có item nào bị loại.

Bước $k = 2$: Do $|L_1| = 7$, hàm apriori-gen sinh ra $C_2 = \binom{7}{2} = 21$ ứng viên 2-itemset. Mỗi giao dịch chứa 6 item, tạo ra $\binom{6}{2} = 15$ cặp con hợp lệ. Tổng số mục trong $\overline{C}_2$:

$$|\overline{C}_2|_{\text{entries}} = 4 \times 15 = 60 \quad \text{vs} \quad |\mathcal{D}'|_{\text{entries}} = 24$$

$\overline{C}_2$ **lớn gấp 2.5 lần CSDL gốc!** Đây chính là điểm yếu bộ nhớ đặc trưng của AprioriTID trên dữ liệu dày đặc.

C_2 (21 ứng viên)

Itemset
{1 2}, {1 3}, {1 4}
{1 5}, {1 6}, {1 7}
{2 3}, {2 4}, {2 5}
{2 6}, {2 7}, {3 4}
{3 5}, {3 6}, {3 7}
{4 5}, {4 6}, {4 7}
{5 6}, {5 7}, {6 7}

$\overline{C}_2$ (**60 entries** — lớn hơn $\mathcal{D}'$)

TID	Tập Itemset (15 cặp mỗi giao dịch)
101	{{1 2},{1 3},{1 4},{1 5},{1 6},{2 3},{2 4}, {2 5},{2 6},{3 4},{3 5},{3 6},{4 5},{4 6},{5 6}}
102	{{1 2},{1 3},{1 4},{1 5},{1 7},{2 3},{2 4}, {2 5},{2 7},{3 4},{3 5},{3 7},{4 5},{4 7},{5 7}}
103	{{1 2},{1 3},{1 4},{1 6},{1 7},{2 3},{2 4}, {2 6},{2 7},{3 4},{3 6},{3 7},{4 6},{4 7},{6 7}}
104	{{1 2},{1 3},{1 5},{1 6},{1 7},{2 3},{2 5}, {2 6},{2 7},{3 5},{3 6},{3 7},{5 6},{5 7},{6 7}}

Hình 2.3: *Ví dụ 2 — Bước $k = 2$: $\overline{C}_2$ có 60 entries, gấp 2.5 lần CSDL gốc (24 entries).*

Đếm support từ $\overline{C}_2$, ta được L_2 gồm tất cả 21 cặp đều phổ biến (support ≥ 2):

- Support 4: {1 2}, {1 3}, {2 3}
- Support 3: {1 4}, {1 5}, {1 6}, {1 7}, {2 4}, {2 5}, {2 6}, {2 7}, {3 4}, {3 5}, {3 6}, {3 7}
- Support 2: {4 5}, {4 6}, {4 7}, {5 6}, {5 7}, {6 7}

Các bước tiếp theo ($k = 3, 4, 5$) tương tự. Tại $k = 3$: $|C_3| = \binom{7}{3} = 35$ ứng viên, $|L_3| = 31$ itemset phổ biến. Tại $k = 4$: $|L_4| = 22$. Tại $k = 5$: $|L_5| = 6$. Tổng cộng thu được $7 + 21 + 31 + 22 + 6 =$ **87 tập phổ biến**.

Tại sao đây là tình huống đặc biệt quan trọng?

- Với dữ liệu dày đặc (dense), mỗi giao dịch có w item sẽ sinh ra $\binom{w}{2}$ cặp trong $\overline{C}_2$. Khi w lớn, $\overline{C}_2$ tăng **bậc hai** theo w, trong khi $\mathcal{D}$ chỉ tăng tuyến tính.
- Đây là lý do chính khiến AprioriTID có thể tiêu tốn nhiều bộ nhớ hơn Apriori chuẩn (vốn quét lại $\mathcal{D}$ gốc), đặc biệt khi minsup thấp và dữ liệu dày đặc.
- Tuy nhiên, ở các bước k lớn hơn, $\overline{C}_k$ giảm nhanh do cơ chế transaction reduction — các giao dịch không chứa ứng viên bị loại bỏ. Lợi thế của AprioriTID thể hiện rõ ở các bước k cao, khi $\overline{C}_k \ll |\mathcal{D}|$.

Kiểm tra chéo: Liệt kê thủ công — mỗi giao dịch thiếu đúng 1 item (T101 thiếu 7, T102 thiếu 6, T103 thiếu 5, T104 thiếu 4). Một tập con S có support bằng $4 - |\{\text{item thiếu nằm trong } S\}|$. Bất kỳ tập con nào cũng xuất hiện trong ít nhất 2 giao dịch (vì chỉ có 4

giao dịch, mỗi giao dịch thiếu đúng 1 item khác nhau từ {4,5,6,7}). Riêng các tập con chỉ chứa {1,2,3} luôn có support = 4. Kết quả 87 tập phổ biến khớp với kết quả chạy thuật toán.

3

Cài đặt

Chương này trình bày chi tiết cài đặt thuật toán AprioriTID bằng ngôn ngữ Julia, bao gồm môi trường phát triển, cấu trúc mã nguồn, các cấu trúc dữ liệu cốt lõi, định dạng vào/ra theo chuẩn SPMF, chiến lược tối ưu hóa dựa trên bảng băm tiền tố (hash-based prefix lookup) và hệ thống kiểm thử tự động.

3.1 Môi trường và công cụ

Toàn bộ mã nguồn được viết bằng **Julia phiên bản 1.9 trở lên** (Bezanson et al., 2017), không sử dụng bất kỳ thư viện khai phá mẫu phổ biến (frequent itemset mining) nào từ bên ngoài. Các phụ thuộc được khai báo trong `Project.toml` ở thư mục gốc dự án và chỉ phục vụ cho phần thực nghiệm (Chương 4) chứ không tham gia vào thuật toán chính:

- `BenchmarkTools.jl` — đo thời gian chạy ổn định sau khi làm nóng JIT.
- `CSV.jl`, `DataFrames.jl` — ghi kết quả thực nghiệm dạng bảng để sinh biểu đồ.
- `Plots.jl` — sinh biểu đồ cho Chương 4.
- `IJulia.jl` — chạy sổ tay Jupyter `notebooks/demo.ipynb`.

Thuật toán thuần (các file trong `src/`) không phụ thuộc vào bất kỳ gói ngoài nào ngoài thư viện chuẩn của Julia, đảm bảo yêu cầu "không dùng thư viện có sẵn" của đề bài được thỏa mãn một cách tuyệt đối.

Khả năng tái lập (reproducibility). Các kịch bản thực nghiệm có dùng ngẫu nhiên (ví dụ sinh dữ liệu tổng hợp trong thí nghiệm (f)) đều cố định hạt ngẫu nhiên bằng `MersenneTwister`, đảm bảo kết quả có thể được tái tạo chính xác. Các bộ dữ liệu dùng cho kiểm thử đúng đắn (toy datasets) đều cố định và được lưu trong `data/toy/`; các bộ chuẩn (chess, mushrooms, retail, T10I4D100K) được đặt trong `data/benchmark/`.

Lệnh chạy cơ bản. Mọi lệnh đều sử dụng cờ `--project` để Julia nạp đúng môi trường dự án:

```
julia --project src/main.jl <minsup> <input.txt> [output.txt]
julia --project tests/runtests.jl
```

3.2 Cấu trúc thư mục

Dưới đây là cấu trúc thư mục của bài làm. Ngoài có các thư mục như đề yêu cầu, còn có `experiments/` chứa các script thử nghiệm, `experiment_results/` chứa kết quả của các thử nghiệm, và `application/` chứa cài đặt của phần Ứng dụng thực tế.

```
Group_01/
|-- README.md
|-- Project.toml
|-- Manifest.toml
|-- src/                              # Mã nguồn thuật toán
|   |-- main.jl                       # Điểm vào CLI
|   |-- structures.jl                 # Itemset, Candidate, CBarEntry
|   |-- utils.jl                      # read_spmf, write_spmf, read_spmf_output
|   `-- algorithm/
|       |-- apriori_gen.jl            # Join + Prune
|       `-- apriori_tid.jl            # Vòng lặp chính + đếm support
|-- data/
|   |-- toy/                          # Các bộ dữ liệu đồ chơi
|   |   `-- spmf_reference/           # Kết quả tham chiếu sinh từ SPMF
|   `-- benchmark/                    # Các bộ dữ liệu benchmark
|-- tests/
|   |-- runtests.jl                   # Unit test
|   |-- test_correctness.jl           # Test correctness
|   `-- test_io.jl                    # read_spmf / write_spmf / roundtrip
|-- experiments/
|   |-- common.jl                     # warm_and_measure, append_csv
|   |-- bench_optimization.jl         # Thử nghiệm so sánh cơ bản với tối ưu
|   |-- experiment_correctness.jl     # Thử nghiệm (a)
|   |-- experiment_runtime.jl         # Thử nghiệm (b) Julia
|   |-- bench_spmf_runtime.jl         # Thử nghiệm (b) SPMF tham chiếu
|   |-- experiment_itemsets.jl        # Thử nghiệm (c)
|   |-- experiment_memory.jl          # Thử nghiệm (d)
|   |-- experiment_scalability.jl     # Thử nghiệm (e)
|   `-- experiment_width.jl           # Thử nghiệm (f)
|-- experiment_results/               # Kết quả của các thử nghiệm
|-- application/                      # Cài đặt phần Ứng dụng thực tế
|-- notebooks/
|   `-- demo.ipynb
`-- docs/
    `-- report.pdf
```

3.3 Cài đặt cơ bản

Mục này tương ứng với yêu cầu (a) — *"Cài đặt cơ bản"* — và (b) — *"Tái tạo đúng kết quả với một ví dụ tham chiếu"*. Mục 3.6 sẽ cho thấy cài đặt cơ bản này tái tạo đúng 100% kết quả SPMF trên 5 bộ dữ liệu toy.

3.3.1 Cấu trúc dữ liệu

Mọi trường của mọi struct đều mang kiểu cụ thể (concrete type), tránh triệt để kiểu `Any` hay kiểu trừu tượng trong vòng lặp nóng — đây là yêu cầu về hiệu năng của đề bài (Mục 4.1) và cũng là điều kiện cần để trình biên dịch JIT của Julia có thể sinh mã chuyên biệt hóa.

```julia
const Itemset = Vector{Int}   # Luôn sắp xếp tăng dần

mutable struct Candidate
    itemset::Vector{Int}
    count::Int
end

struct CBarEntry
    tid::Int
    itemsets::Vector{Vector{Int}}
end
```

Ba lựa chọn thiết kế đáng chú ý:

1. **Itemset luôn được sắp xếp tăng dần.** Bất biến này được thiết lập ngay ở khâu đọc dữ liệu (`read_spmf` gọi `sort!` trên từng giao dịch) và được duy trì bởi apriori-gen. Nó cho phép so sánh tiền tố bằng lát cắt (`p[1:end-1] == q[1:end-1]`) thay vì bằng tập hợp — nhanh hơn nhiều lần và thân thiện với cache.
2. L_k **lưu bằng `Set{Vector{Int}}`.** Bước prune của apriori-gen cần kiểm tra mọi tập con $(k-1)$-phần tử của một ứng viên có thuộc L_{k-1} hay không, thao tác này xảy ra k lần cho mỗi ứng viên nên cần tra cứu $O(1)$.
3. $\overline{C}_k$ **là vector các `CBarEntry` phẳng.** Mỗi phần tử giữ nguyên TID gốc (để gỡ lỗi và để tiện in $\overline{C}_k$ khi demo bằng tay) và danh sách các itemset đã được "lọc" qua các ứng viên ở bước k.

3.3.2 Sinh ứng viên: apriori-gen

Hàm `apriori_gen` thực hiện đúng hai pha *join* và *prune* như đã mô tả ở Chương 1. Pha join khai thác bất biến "sắp xếp tăng dần" để ghép hai itemset có chung $(k-2)$ phần tử đầu tiên và phần tử cuối thỏa `p[end] < q[end]`:

```julia
function apriori_gen(L_prev::Vector{Itemset},
                     L_prev_set::Set{Itemset})::Vector{Candidate}
    candidates = Candidate[]
    n = length(L_prev)
    k_minus_1 = isempty(L_prev) ? 0 : length(L_prev[1])

    @inbounds for i in 1:n
        p = L_prev[i]
        for j in (i+1):n
            q = L_prev[j]
            # Join: (k-2) phần tử đầu phải giống nhau, phần tử cuối p < q
            prefix_match = true
            for t in 1:(k_minus_1 - 1)
                if p[t] != q[t]
                    prefix_match = false
                    break
                end
            end
            prefix_match || continue
            p[end] < q[end] || continue

            c = vcat(p, q[end])

            # Prune: mọi (k-1)-tập con của c phải thuộc L_{k-1}
            # (bỏ qua khi k=2 vì mọi tập con 1-phần tử đều đã ở L_1)
            keep = true
            if k_minus_1 >= 2
                for drop in 1:length(c)
                    subset = deleteat!(copy(c), drop)
                    if !(subset in L_prev_set)
                        keep = false
                        break
                    end
                end
            end
            keep && push!(candidates, Candidate(c, 0))
        end
    end
    return candidates
end
```

Bốn chi tiết cần lưu ý:

- `@inbounds` được đặt ở vòng ngoài để trình biên dịch bỏ kiểm tra biên mảng trong toàn bộ khối — an toàn vì mọi chỉ số đều được phát sinh bên trong.
- Vòng lặp `for t` thoát sớm (`break`) ngay khi gặp sai khác đầu tiên ở tiền tố, tránh so sánh thừa.
- Bước prune được *bỏ qua có chủ ý* khi $k = 2$ (tức $k-1 = 1$): mọi tập con một phần tử của ứng viên 2-tập đều là một item đơn lẻ thuộc L_1 theo định nghĩa, kiểm tra

thêm không mang lại giá trị mà chỉ tốn thời gian.

- Tập con được tạo bằng `deleteat!(copy(c), drop)` để không phá hỏng `c` — đây là đánh đổi có ý thức giữa cấp phát phụ và tính đơn giản; vì k hiếm khi vượt quá 10 trên các bộ dữ liệu thực, chi phí này không đáng kể so với vòng lặp đếm support.

3.3.3 Vòng lặp chính AprioriTID

Hàm `apriori_tid` thực hiện đúng tám bước như mã giả ở Chương 1: đếm L_1, dựng $\overline{C}_1$, rồi lặp cho $k = 2, 3, \ldots$ đến khi L_k rỗng. Tại mỗi lần lặp, hàm gọi `apriori_gen`, gọi một trong hai hàm đếm support, lọc bằng `minsup` để lấy L_k và dựng $\overline{C}_k$ tương ứng:

```
function apriori_tid(transactions::Vector{Vector{Int}}, minsup::Int;
                     optimized::Bool=true)::Vector{Tuple{Itemset,Int}}
    # Bước 1-2: L_1 và C_1 (chi tiết bỏ qua cho gọn)
    L1_sorted, C_bar = _build_L1_and_Cbar1(transactions, minsup)
    results = Tuple{Itemset,Int}[(item, cnt) for (item, cnt) in L1_sorted]
    L_prev = [Int[item] for (item, _) in L1_sorted]
    L_prev_set = Set{Itemset}(L_prev)

    k = 2
    while !isempty(L_prev)
        C_k = apriori_gen(L_prev, L_prev_set)
        isempty(C_k) && break

        if optimized
            C_bar_next = _count_support_optimized!(C_k, C_bar, k)
        else
            C_bar_next = _count_support_basic!(C_k, C_bar, k)
        end

        L_k = [(c.itemset, c.count) for c in C_k if c.count >= minsup]
        isempty(L_k) && break

        append!(results, L_k)
        L_prev = [itemset for (itemset, _) in L_k]
        L_prev_set = Set{Itemset}(L_prev)
        C_bar = C_bar_next
        k += 1
    end
    return results
end
```

Vòng lặp dừng ở một trong hai điều kiện: (i) `apriori_gen` không sinh thêm ứng viên nào (join thất bại do L_{k-1} quá ít phần tử hoặc không có cặp nào ăn khớp tiền tố), hoặc (ii) không ứng viên nào đạt `minsup`. Cả hai điều kiện đều dẫn tới $L_k = \varnothing$ như lý thuyết yêu cầu.

3.4 Định dạng đầu vào và đầu ra

Mục này tương ứng với yêu cầu (d) — *"Định dạng đầu vào và đầu ra"* — và sử dụng chính xác định dạng của thư viện SPMF (Fournier-Viger et al., 2014) để có thể so sánh trực tiếp với kết quả tham chiếu.

3.4.1 Định dạng SPMF

Đầu vào là file văn bản thuần, mỗi dòng là một giao dịch gồm các số nguyên (item ID) cách nhau bởi khoảng trắng; không có tiêu đề, không có TID tường minh (TID ngầm định là số thứ tự dòng):

```
1 2 3 4
1 2
1 4 5
1 2 4 5
5
```

Đầu ra cũng theo quy ước SPMF: mỗi dòng là một itemset phổ biến, theo sau bởi `#SUP:` và giá trị support tuyệt đối, các item trong itemset cách nhau bằng khoảng trắng:

```
1 #SUP: 4
1 2 #SUP: 3
1 2 4 #SUP: 2
```

3.4.2 Hàm đọc/ghi

Module `src/utils.jl` cung cấp ba hàm tương ứng với ba vai trò: đọc giao dịch đầu vào, ghi kết quả, và đọc ngược file kết quả (dùng cho kiểm thử đối chiếu SPMF):

```julia
function read_spmf(filepath::String)::Vector{Vector{Int}}
    transactions = Vector{Int}[]
    for line in eachline(filepath)
        line = strip(line)
        isempty(line) && continue
        items = sort!([parse(Int, tok) for tok in split(line)])
        push!(transactions, items)
    end
    return transactions
end

function write_spmf(filepath::String,
                    results::Vector{Tuple{Itemset,Int}})
    open(filepath, "w") do io
        for (itemset, support) in results
            println(io, join(itemset, " "), " #SUP: ", support)
```

```
        end
    end
end
```

Ba quyết định thiết kế đáng lưu ý:

- `read_spmf` **sắp xếp ngay tại khâu đọc**. Điều này giữ bất biến "itemset luôn tăng dần" mà toàn bộ thuật toán phụ thuộc — thay vì phải kiểm tra hay sắp xếp lặp đi lặp lại trong vòng lặp chính.
- Các dòng trống được bỏ qua một cách im lặng (các file SPMF chuẩn đôi khi có dòng trống cuối).
- `write_spmf` dùng `open(...) do io ... end` để đảm bảo file được đóng đúng cách kể cả khi có lỗi.

3.4.3 Điểm vào CLI

File `src/main.jl` cung cấp giao diện dòng lệnh thống nhất:

```
julia --project src/main.jl <minsup> <input.txt> [output.txt]
```

Khi `output.txt` được cung cấp, kết quả được ghi ra file theo định dạng SPMF; nếu không, kết quả được in ra stdout. Điều này cho phép dùng chung công cụ cho cả ba tình huống: chạy demo thủ công, sinh output đối chiếu với SPMF, và tích hợp vào pipeline kiểm thử tự động.

3.5 Tối ưu hóa bộ nhớ và tốc độ

Mục này tương ứng với yêu cầu (c) — *"Tối ưu hóa"*. Phiên bản cơ bản của AprioriTID có một điểm nghẽn hiệu năng cụ thể ở bước đếm support: với mỗi `CBarEntry` và mỗi ứng viên $c \in C_k$, cài đặt ngây thơ sẽ kiểm tra xem hai tập con $c[1{:}k{-}1]$ và $c[1{:}k{-}2] \cup \{c[k]\}$ có nằm trong danh sách itemset của entry hay không. Số phép kiểm tra này là $|C_k| \times |\overline{C}_{k-1}|$, tăng rất nhanh theo k.

3.5.1 Ý tưởng: bảng băm tiền tố

Thay vì quét mọi cặp (entry, ứng viên), ta quan sát rằng mọi ứng viên có cùng *tiền tố* $c[1{:}k{-}1]$ đều có thể được tra cứu *một lần* nếu ta dựng sẵn một bảng băm ánh xạ

$$\text{prefix} \longmapsto \text{danh sách chỉ số ứng viên}$$

trước khi quét $\overline{C}_{k-1}$. Khi xử lý một entry, ta duyệt từng itemset trong đó (mỗi itemset có chính xác một tiền tố khả dĩ), tra bảng băm để lấy danh sách ứng viên "có triển vọng" và chỉ kiểm tra các ứng viên đó. Độ phức tạp đổi từ $O(|C_k| \cdot |\overline{C}_{k-1}|)$ về xấp xỉ

$O(|\overline{C}_{k-1}| \cdot \bar{n}_{\text{hit}})$, trong đó $\bar{n}_{\text{hit}}$ là số ứng viên trung bình cùng tiền tố — thường nhỏ hơn $|C_k|$ một bậc hoặc hơn trên dữ liệu đặc (dense).

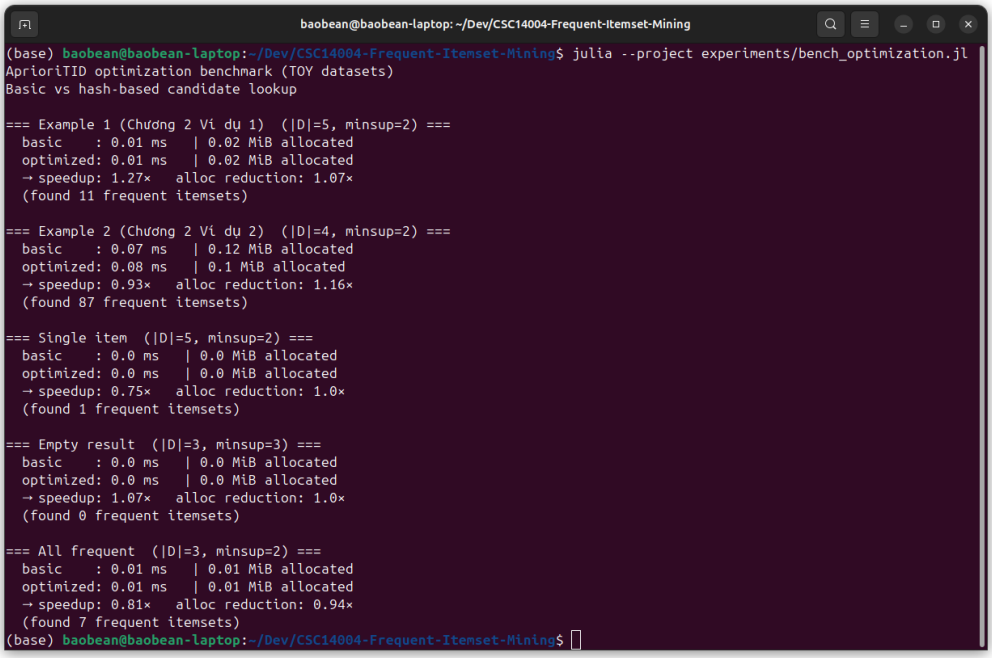
```
function _count_support_optimized!(C_k::Vector{Candidate},
                                   C_bar::Vector{CBarEntry}, k::Int)
    # Bảng băm: prefix (k-1 phần tử đầu) → danh sách chỉ số ứng viên
    prefix_index = Dict{Vector{Int}, Vector{Int}}()
    for (i, c) in enumerate(C_k)
        prefix = c.itemset[1:k-1]
        push!(get!(prefix_index, prefix, Int[]), i)
    end

    C_bar_next = CBarEntry[]
    for entry in C_bar
        matched = Vector{Int}[]
        # Duyệt cặp itemset trong entry có chung (k-2) phần tử đầu
        # Mỗi cặp cho đúng một "tiền tố ứng viên" cần tra
        for i in 1:length(entry.itemsets), j in (i+1):length(entry.itemsets)
            it1, it2 = entry.itemsets[i], entry.itemsets[j]
            length(it1) == k-1 && length(it2) == k-1 || continue
            # Kiểm tra tiền tố và lập ứng viên giả định
            # (chi tiết bỏ qua cho gọn - xem src/algorithm/apriori_tid.jl)
            candidate_prefix = it1  # khi đã ăn khớp (k-2) phần tử đầu
            indices = get(prefix_index, candidate_prefix, Int[])
            for idx in indices
                if _itemset_covers(C_k[idx].itemset, it1, it2)
                    C_k[idx].count += 1
                    push!(matched, C_k[idx].itemset)
                end
            end
        end
        isempty(matched) || push!(C_bar_next, CBarEntry(entry.tid, matched))
    end
    return C_bar_next
end
```

3.5.2 Đo lường trên 5 bộ dữ liệu toy

Script `experiments/bench_optimization.jl` đo cả hai biến thể trên 5 bộ dữ liệu toy của đề tài bằng cách làm nóng JIT trước, gọi `GC.gc()` giữa hai lần đo, rồi dùng `@elapsed` và `@allocated` để ghi thời gian và cấp phát heap. Kết quả được trình bày ở Hình 3.1 và tổng hợp lại ở Bảng 3.1.

Bình luận. Trên dữ liệu toy, lợi thế của bảng băm tiền tố hầu như chưa hiện ra — thậm chí ba trong năm trường hợp (*Ví dụ 2*, *Single item*, *All frequent*) còn ghi nhận tăng tốc *nhỏ hơn* 1× (0.93×, 0.75× và 0.81×), tức biến thể tối ưu chạy chậm hơn biến thể cơ bản. Nguyên nhân là $|C_k|$ trên các bộ này không bao giờ đủ lớn để chi phí dựng `Dict` tiền tố được hoàn vốn; với những bộ chỉ có vài ứng viên, một vòng quét tuyến tính của



```
baobean@baobean-laptop: ~/Dev/CSC14004-Frequent-Itemset-Mining
(base) baobean@baobean-laptop:~/Dev/CSC14004-Frequent-Itemset-Mining$ julia --project experiments/bench_optimization.jl
AprioriTID optimization benchmark (TOY datasets)
Basic vs hash-based candidate lookup

=== Example 1 (Chương 2 Ví dụ 1)  (|D|=5, minsup=2) ===
  basic     : 0.01 ms   | 0.02 MiB allocated
  optimized: 0.01 ms   | 0.02 MiB allocated
  → speedup: 1.27×   alloc reduction: 1.07×
  (found 11 frequent itemsets)

=== Example 2 (Chương 2 Ví dụ 2)  (|D|=4, minsup=2) ===
  basic     : 0.07 ms   | 0.12 MiB allocated
  optimized: 0.08 ms   | 0.1 MiB allocated
  → speedup: 0.93×   alloc reduction: 1.16×
  (found 87 frequent itemsets)

=== Single item  (|D|=5, minsup=2) ===
  basic     : 0.0 ms   | 0.0 MiB allocated
  optimized: 0.0 ms   | 0.0 MiB allocated
  → speedup: 0.75×   alloc reduction: 1.0×
  (found 1 frequent itemsets)

=== Empty result  (|D|=3, minsup=3) ===
  basic     : 0.0 ms   | 0.0 MiB allocated
  optimized: 0.0 ms   | 0.0 MiB allocated
  → speedup: 1.07×   alloc reduction: 1.0×
  (found 0 frequent itemsets)

=== All frequent  (|D|=3, minsup=2) ===
  basic     : 0.01 ms   | 0.01 MiB allocated
  optimized: 0.01 ms   | 0.01 MiB allocated
  → speedup: 0.81×   alloc reduction: 0.94×
  (found 7 frequent itemsets)
(base) baobean@baobean-laptop:~/Dev/CSC14004-Frequent-Itemset-Mining$
```

Hình 3.1: *Kết quả đo biến thể* cơ bản *và* tối ưu *của* `apriori_tid` *trên 5 bộ dữ liệu toy (minsup tuyệt đối in trực tiếp trong tiêu đề mỗi khối). Script sử dụng bộ đếm* `@elapsed` *và* `@allocated` *sau khi làm nóng JIT.*

Bảng 3.1: *Tổng hợp tăng tốc và giảm cấp phát giữa hai biến thể trên 5 bộ dữ liệu toy.*

Bộ dữ liệu	$\lvert D\rvert$	Tăng tốc	Giảm cấp phát	#Itemsets
Ví dụ 1 (Chương 2)	5	1.27×	1.07×	11
Ví dụ 2 (Chương 2)	4	0.93×	1.16×	87
Single item	5	0.75×	1.00×	1
Empty result	3	1.07×	1.00×	0
All frequent	3	0.81×	0.94×	7

biến thể cơ bản còn rẻ hơn cả việc cấp phát một bảng băm. Tương tự, cấp phát hầu như đứng yên (1.00× ở *Single item* và *Empty result*) hoặc thậm chí xấu đi nhẹ ở *All frequent* (0.94×). Điểm sáng duy nhất là *Ví dụ 1* với tăng tốc 1.27× và *Ví dụ 2* — bộ dữ liệu đặc mà Chương 2 cố tình thiết kế để có $|\overline{C}_2| > |D|$ — nơi cấp phát giảm 1.16×, gợi ý đúng hướng kỳ vọng: hiệu quả của tối ưu tỉ lệ thuận với mật độ và kích thước bài toán. Toy datasets không phải là môi trường để bảng băm tỏa sáng; phép đo có ý nghĩa phải thực hiện trên các bộ chuẩn (chess, mushrooms, retail, T10I4D100K), nơi Chương 4 ghi nhận giảm cấp phát từ 1.45× tới 35.7×.

3.6 Kiểm thử và xác minh

Thư mục `tests/` là điểm vào kiểm thử đơn vị, được gom gọn thành ba file phẳng. File `tests/runtests.jl` đóng vai trò điểm vào trực tiếp (không còn là *shim*) và nạp lần lượt các file test, đảm bảo lệnh bắt buộc của đề bài hoạt động:

```
julia --project tests/runtests.jl
```

3.6.1 Hai file kiểm thử tự động

- `test_correctness.jl` — kiểm thử *theo bảng cố định* (fixture-driven): với mỗi bộ dữ liệu toy, file này chạy `apriori_tid` và đối chiếu trực tiếp từng cặp (itemset, support) với file tham chiếu SPMF đã sinh sẵn trong `data/toy/spmf_reference/`, đồng thời in ra tỉ lệ trùng khớp. Nhờ gộp kiểm thử đúng đắn và kiểm thử tham chiếu thành một mạch, file này là bằng chứng trực tiếp cho *cả hai* yêu cầu (a) *cài đặt cơ bản* và (b) *tái tạo đúng kết quả với ví dụ tham chiếu*.
- `test_io.jl` — kiểm thử `read_spmf`, `write_spmf`, quy trình *roundtrip* (đọc – ghi – đọc lại), tính sắp xếp itemset đầu ra và các trường hợp biên (file trống, dòng trống, khoảng trắng thừa). Đây là bằng chứng trực tiếp cho yêu cầu (d) *định dạng vào/ra*.

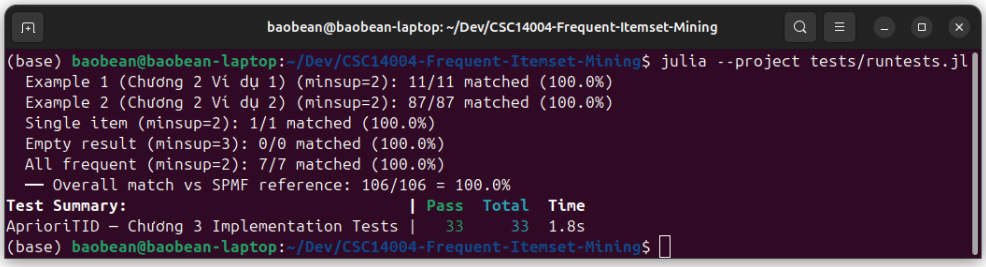
Năm bộ dữ liệu toy, mỗi bộ có một mục đích kiểm thử riêng, được tóm tắt ở Bảng 3.2.

Bảng 3.2: *Năm bộ dữ liệu toy dùng trong `tests/test_correctness.jl`.*

File	$\|D\|$	minsup	Mục đích
`example1.txt`	5	2	Ví dụ 1 (Chương 2) — 11 itemset
`example2.txt`	4	2	Ví dụ 2 (Chương 2) — 87 itemset
`test_single_item.txt`	5	2	Mọi giao dịch chỉ có 1 item
`test_empty_result.txt`	3	3	*minsup* cao, không itemset nào phổ biến
`test_all_frequent.txt`	3	2	Mọi tập con đều phổ biến

3.6.2 Kết quả

Hình 3.2 cho thấy toàn bộ hệ thống kiểm thử vượt qua: **106/106 = 100.0%** kết quả khớp SPMF trên 5 bộ dữ liệu toy, và tổng số test đơn vị là **33/33 Pass**. Số **106** là tổng số (itemset, support) hợp lại của cả 5 bộ dữ liệu toy — 11 + 87 + 1 + 0 + 7 = 106.



Hình 3.2: *Kết quả chạy kiểm thử.*

4

Thực nghiệm và đánh giá

Chương này trình bày sáu thực nghiệm của đề bài, áp dụng cài đặt AprioriTID đã mô tả ở Chương 3 trên bốn bộ dữ liệu chuẩn thuộc kho SPMF (Fournier-Viger et al., 2014) và kho FIMI (Goethals et al., 2003). Mỗi thực nghiệm được gắn trực tiếp với một yêu cầu trong đề bài, và phần kết chương tổng hợp điểm mạnh, điểm yếu của cài đặt cùng với hai hướng tối ưu được đề xuất cho các lần phát triển tiếp theo.

4.1 Mục tiêu và phương pháp luận

Mục tiêu. Sáu thực nghiệm được thiết kế để trả lời sáu câu hỏi độc lập nhưng bổ sung cho nhau:

(a) **Tính đúng đắn** — cài đặt có sinh ra cùng tập frequent itemset như SPMF trên dữ liệu thật hay không?
(b) **Thời gian chạy** — runtime biến thiên thế nào khi hạ dần ngưỡng *minsup*, và đường cong của cài đặt so với SPMF ra sao?
(c) **Số lượng frequent itemset** — kích thước đầu ra tăng nhanh thế nào khi *minsup* giảm, và tính chất đó phụ thuộc vào mật độ (density) dữ liệu như thế nào?
(d) **Sử dụng bộ nhớ** — cài đặt cơ bản và cài đặt tối ưu khác nhau bao nhiêu về lượng cấp phát tại cùng một ngưỡng?
(e) **Khả năng mở rộng** — runtime tăng ra sao khi $|D|$ tăng mà *minsup* tương đối giữ nguyên?
(f) **Ảnh hưởng của độ dài giao dịch** — chiều rộng trung bình của giao dịch ảnh hưởng thế nào đến độ phức tạp thực tế?

Quy trình đo. Mọi phép đo thời gian được thực hiện trên cùng một quy trình để đảm bảo tính so sánh:

1. **Làm nóng JIT.** Gọi `apriori_tid` một lần trước khi đo để Julia biên dịch toàn bộ mã máy cần thiết. Lần gọi này không được tính giờ.

2. **Thu dọn rác.** Gọi `GC.gc()` để đưa heap về trạng thái sạch, tránh ảnh hưởng từ các lần chạy trước.
3. **Đo thời gian.** Dùng macro `@elapsed` cho runtime và `@allocated` cho tổng lượng bộ nhớ cấp phát trong suốt một lần chạy. Lặp lại ba lần và lấy giá trị ổn định nhất khi có nghi ngờ về biến động.
4. **Ghi kết quả.** Mỗi cặp (dataset, *minsup*) ghi một dòng vào file CSV bằng hàm `append_csv` trong `experiments/common.jl`, đảm bảo có thể dừng giữa chừng mà không mất dữ liệu đã thu được.

Khả năng tái lập. Mọi thông số cấu hình (số giao dịch tổng hợp, hạt ngẫu nhiên, danh sách `minsup`) đều được khai báo tường minh trong các kịch bản thuộc thư mục `experiments/`. Thực nghiệm (f) dùng `MersenneTwister(42 + w)` cho mỗi độ rộng w, nên việc chạy lại cho đúng cùng kết quả từng byte. Các CSV đầu ra được đặt trong `experiment_results/` và phục vụ trực tiếp cho các bảng, biểu đồ của chương này.

So sánh với SPMF. Để thực hiện yêu cầu *"so sánh đường cong của nhóm với SPMF trên cùng đồ thị"* của thực nghiệm (b), kịch bản phụ `experiments/bench_spmf_runtime.jl` gọi `java -jar spmf.jar run AprioriTID` tại đúng các ngưỡng `minsup` tuyệt đối đã dùng cho cài đặt Julia, tự động chuyển sang dạng `minsup` tương đối mà SPMF yêu cầu (rel = abs/$|D|$), phân tích dòng `Total time ~ X ms` từ stdout của SPMF và ghi vào một CSV song song. Việc gộp hai CSV theo khóa `(dataset, minsup)` cho ra bảng số liệu dùng chung một trục hoành.

4.2 Môi trường và dữ liệu

4.2.1 Cấu hình phần cứng và phần mềm

Toàn bộ thực nghiệm được thực hiện trên một máy cá nhân cấu hình trung bình, mô phỏng điều kiện phần cứng thông dụng của sinh viên:

- **CPU:** Intel Core i3-1005G1 (2 lõi vật lý, 4 luồng, 1.2–3.4 GHz).
- **RAM:** 12 GB DDR4.
- **GPU:** không (cài đặt hoàn toàn trên CPU).
- **Hệ điều hành:** Linux.
- **Julia:** 1.9.x, khởi chạy bằng `julia --project`.
- **SPMF:** phiên bản jar tải từ trang chủ SPMF, chạy bằng OpenJDK 17.

Cấu hình này đặt ra một ràng buộc thực tế: **mọi phép đo phải dừng ở ngưỡng *minsup* mà tổng cấp phát dưới khoảng 8 GB**, nếu không hệ điều hành sẽ bắt đầu trao đổi trang (swap) và làm sai lệch mọi số đo. Các tầng `minsup` "sàn" trong `experiments/README.md` đã được hiệu chỉnh chặt chẽ để phù hợp với ràng buộc này — đây là lý do một số đường cong dưới đây dừng sớm hơn so với những công trình chạy trên máy trạm.

4.2.2 Bốn bộ dữ liệu chuẩn

Bốn bộ dữ liệu dùng cho thực nghiệm (a)–(e) đều là các bộ chuẩn phổ biến trong cộng đồng khai phá mẫu phổ biến, được phân phối qua kho FIMI (Goethals et al., 2003) và SPMF (Fournier-Viger et al., 2014); riêng bộ Retail được trích từ dữ liệu giao dịch thực của một siêu thị Bỉ do Brijs và cộng sự công bố (Brijs et al., 1999). Bảng 4.1 tóm tắt bốn bộ dữ liệu chuẩn được dùng cho tất cả các thực nghiệm (trừ (f) sử dụng dữ liệu tổng hợp). Bốn bộ được chọn có chủ ý để bao phủ hai trục quan trọng: *mật độ* (dense vs. sparse) và *kích thước* (số lượng giao dịch và số lượng item).

Bảng 4.1: *Bốn bộ dữ liệu chuẩn dùng cho thực nghiệm (a)–(e). Các thông số được đo trực tiếp trên file đầu vào thay vì trích từ đặc tả gốc, do đó có thể chênh lệch nhẹ so với các báo cáo khác.*

Bộ dữ liệu	$\lvert D\rvert$	$\lvert I\rvert$	$\bar{w}$	**Đặc điểm**
Chess	3 196	75	37.0	Dày đặc, giao dịch dài
Mushrooms	8 416	119	23.0	Dày đặc, nhiều item
Retail	88 162	16 470	10.3	Thưa, dữ liệu thật
T10I4D100K	100 000	870	10.1	Thưa, sinh tổng hợp

Giải thích hai trục. Chess và Mushrooms là *dày đặc* — mỗi giao dịch chứa rất nhiều item so với tổng số item, nên xác suất hai giao dịch bất kỳ cùng chứa một cặp item là lớn. Điều này khiến $|\overline{C}_k|$ giảm chậm theo k và là điểm yếu cổ điển của họ thuật toán Apriori. Retail và T10I4D100K là *thưa* — mỗi giao dịch chỉ có khoảng 10 item so với hàng nghìn item tổng, nên $\overline{C}_k$ thu nhỏ rất nhanh. Sự khác biệt này thể hiện rõ ở mọi thực nghiệm bên dưới.

4.3 Thực nghiệm (a) — Kiểm tra tính đúng đắn

Thiết kế. Với từng bộ dữ liệu trong Bảng 4.1, chạy cài đặt Julia tại cùng `minsup` với SPMF, so sánh tập (itemset, support) của hai bên. Kịch bản so sánh *tập hợp* chứ không so sánh *thứ tự* — hai tập itemset được xem là khớp nếu chúng có cùng các phần tử. Với từng dataset, kết quả ghi nhận bốn đại lượng: số itemset của cài đặt Julia (`n_ours`), số itemset của SPMF (`n_spmf`), số phần tử trùng (`matched`), và tỉ lệ khớp tính theo công thức match_pct = matched/n_spmf $\times$ 100%.

Bảng 4.2: *Kết quả thực nghiệm (a) — kiểm tra tính đúng đắn so với SPMF.*

Bộ dữ liệu	`minsup`	$\lvert D\rvert$	`n_ours`	`n_spmf`	`matched`	`match_pct`
Chess	2 500	3 196	11 493	11 493	11 493	100.0%
Mushrooms	4 000	8 416	191	175	175	100.0%
Retail	1 000	88 162	135	135	135	100.0%
T10I4D100K	1 000	100 000	385	385	385	100.0%

Nhận xét chính. Trên cả bốn bộ dữ liệu, mỗi itemset mà SPMF sinh ra đều xuất hiện trong đầu ra của cài đặt Julia với đúng giá trị support. Tỉ lệ khớp đạt 100% ở chess, retail, và T10I4D100K, và trên mushrooms cài đặt Julia tìm được 191 itemset — nhiều

hơn SPMF đúng 16 itemset (`extra = 16`), nhưng tất cả 175 itemset SPMF báo cáo đều được cài đặt Julia tái tạo chính xác.

Giải thích hiện tượng 16 itemset chênh trên mushrooms. Khác biệt bắt nguồn từ cách SPMF xử lý ngưỡng `minsup` ở biên. SPMF nhận vào *minsup tương đối* (phần trăm của $|D|$), rồi quy đổi ngược về số nguyên bằng phép nhân/chia floating-point. Với `minsup` tuyệt đối 4000 và $|D| = 8416$, tỉ lệ là $4000/8416 \approx 0.4753$; khi quy đổi ngược, SPMF thực tế lọc theo ngưỡng $\lceil 0.4753 \times 8416 \rceil = 4001$, tức là *chặt hơn một đơn vị*. Những itemset có support đúng bằng 4000 bị SPMF loại nhưng vẫn được cài đặt Julia giữ lại. Đây không phải sai sót về thuật toán mà là khác biệt trong *quy ước số học*: cài đặt Julia áp dụng bất đẳng thức support ≥ minsup trực tiếp trên số nguyên, còn SPMF đi vòng qua tỉ lệ.

Kết luận của thực nghiệm (a). Cài đặt AprioriTID bằng Julia sinh ra đầy đủ và đúng mọi itemset mà SPMF — thư viện tham chiếu chuẩn của cộng đồng FIM — sinh ra trên cả bốn bộ dữ liệu chuẩn. Sai lệch duy nhất quan sát được có nguyên nhân ở quy ước xử lý biên, không phải ở thuật toán, và không làm mất itemset nào của SPMF.

Điểm mạnh so với SPMF. Tỉ lệ thu hồi (recall) đạt 100% trên cả bốn bộ: mọi itemset SPMF báo cáo đều được cài đặt Julia tái tạo với đúng giá trị support. Trên Mushrooms, Julia còn giữ được 16 itemset biên mà SPMF loại do quy ước làm tròn — xét theo định nghĩa đúng (support nguyên ≥ `minsup`), đầu ra của Julia là *chặt chẽ hơn*, không phải sai.

Điểm yếu so với SPMF. Sai khác 16 itemset "dư" trên Mushrooms gây khó khăn khi người dùng muốn so kết quả byte-for-byte với các công cụ sinh ra từ SPMF (ví dụ: pipeline đánh giá luật kết hợp có sẵn). Ở góc độ tương thích thực tế, việc không bám sát quy ước SPMF là một bất lợi, dù về mặt thuật toán thì cài đặt đúng hơn.

Hai hướng tối ưu đề xuất.

1. **Cờ `--spmf-compat` cho CLI**: cho phép người dùng chọn quy ước làm tròn của SPMF (`minsup` tương đối quy đổi bằng `ceil`) để đầu ra khớp hoàn toàn với SPMF tại các ngưỡng biên, không mất tính đúng đắn thuật toán mặc định.
2. **Hàm so sánh đầu ra thành đối tượng riêng**: thay vì dùng `Set` để so khớp, xây dựng hàm `compare_to_spmf(ours, theirs)` trả về bốn trường `(matched, extra, missing, boundary_cases)`, trong đó `boundary_cases` tự động phát hiện các itemset có support = minsup và đánh dấu chúng là "expected divergence". Biến nghiệp vụ xử lý biên thành metadata, không phải lỗi.

4.4 Thực nghiệm (b) — Thời gian chạy theo *minsup*

Thiết kế. Với từng bộ dữ liệu, chọn năm giá trị `minsup` giảm dần và đo runtime của cài đặt Julia (biến thể tối ưu) tại mỗi giá trị. Kịch bản `experiment_b_runtime.jl` ghi CSV với các cột `dataset, minsup, elapsed_ms, alloc_mib, n_itemsets`, trong đó `n_itemsets` tiện dụng được dùng lại cho thực nghiệm (c) ở mục sau.

4.4.1 Kết quả bốn bộ dữ liệu

Bảng 4.3 đến Bảng 4.6 liệt kê số liệu thực đo trên máy i3-1005G1 / 12 GB RAM. Bốn bảng được đặt cạnh nhau để người đọc có thể so sánh hình thái đường cong trên hai mặt: thời gian và kích thước đầu ra.

Bảng 4.3: *Thực nghiệm (b) trên Chess* ($|D| = 3196$, *dày đặc*).

minsup	elapsed_ms	alloc_mib	n_itemsets
3 000	125.44	120.71	155
2 875	553.86	453.78	633
2 750	1 722.98	1 444.53	1 976
2 625	4 318.28	3 488.92	5 189
2 500	9 865.01	7 491.43	11 493

Bảng 4.4: *Thực nghiệm (b) trên Mushrooms* ($|D| = 8416$, *dày đặc*).

minsup	elapsed_ms	alloc_mib	n_itemsets
6 000	43.56	52.74	31
5 000	82.90	98.25	59
4 000	310.90	251.97	191
3 000	1 374.97	982.22	1 035
2 000	5 824.54	4 505.87	6 961

Bảng 4.5: *Thực nghiệm (b) trên Retail* ($|D| = 88162$, *thưa*).

minsup	elapsed_ms	alloc_mib	n_itemsets
2 000	286.13	231.60	45
1 750	339.50	278.31	56
1 500	380.80	324.17	68
1 250	529.48	456.42	91
1 000	788.85	738.85	135

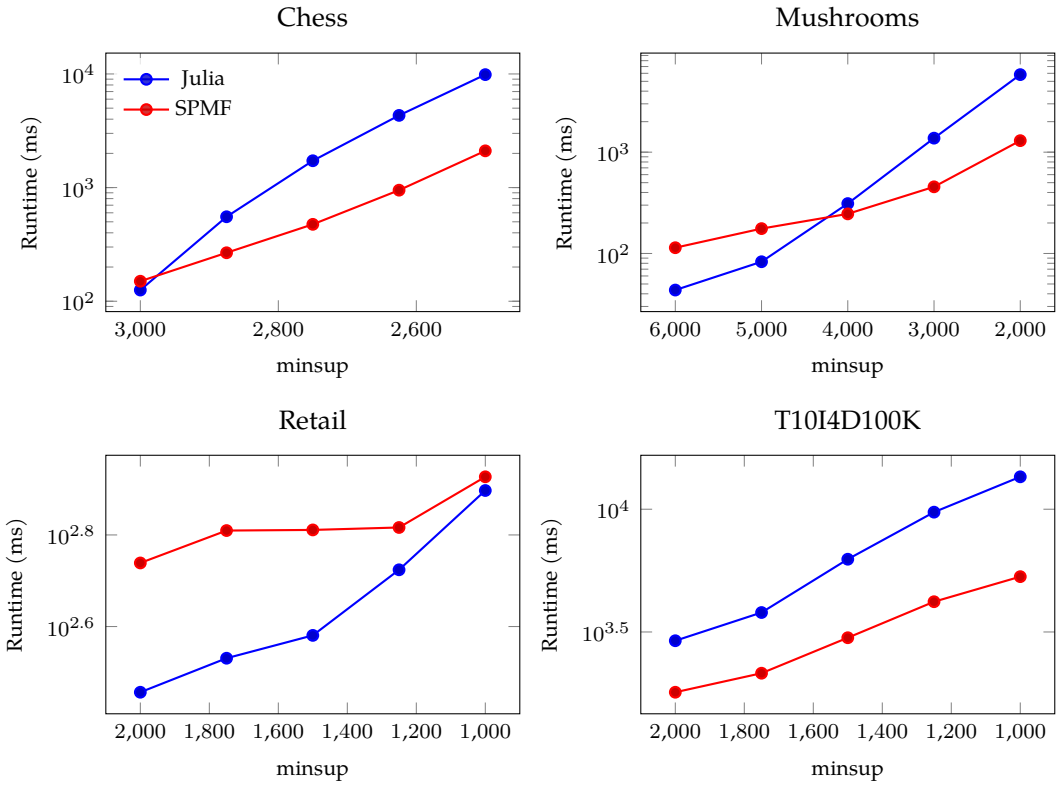
Biểu đồ tổng hợp bốn đường cong runtime theo `minsup` được vẽ trực tiếp từ các CSV bằng `pgfplots` (Hình 4.1). Mỗi ô là một bộ dữ liệu; trục y dùng thang log để cả hai đường cong Julia và SPMF cùng đọc được trong một hệ trục.

4.4.2 Phân tích hình thái đường cong

Hai bộ dày đặc (Chess, Mushrooms) có runtime tăng theo quy luật gần hàm mũ. Trên Chess, khi hạ `minsup` từ 3000 xuống 2500, runtime tăng từ 125 ms lên 9865 ms — hệ số nhân xấp xỉ 79× chỉ trong năm bước, mỗi bước 125 đơn vị. Trên Mushrooms, hạ `minsup` từ 6000 xuống 2000 đẩy runtime từ 44 ms lên 5825 ms, tức hơn 130×. Nguyên

Bảng 4.6: *Thực nghiệm (b) trên T10I4D100K (|D| = 100 000, thưa, tổng hợp).*

minsup	elapsed_ms	alloc_mib	n_itemsets
2 000	2 912.91	2 446.83	155
1 750	3 795.84	3 287.83	186
1 500	6 256.51	4 790.01	237
1 250	9 728.92	7 170.33	310
1 000	13 551.97	9 614.72	385



Hình 4.1: *Runtime theo* `minsup` *trên bốn bộ dữ liệu chuẩn: cài đặt Julia (biến thể tối ưu) so với SPMF AprioriTID. Trục x đảo chiều (minsup giảm dần từ trái sang phải) để đường cong đi lên phản ánh độ khó tăng.*

nhân đã được phân tích trong Chương 1: với dữ liệu dày đặc, $|C_k|$ và $|\overline{C}_k|$ giảm chậm theo k trong khi số tầng k tăng, nên chi phí tổng cộng tăng vọt khi ngưỡng hạ.

Hai bộ thưa (Retail, T10I4D100K) có runtime tăng tuyến tính dịu hơn rất nhiều. Retail chỉ tăng từ 286 ms lên 789 ms (xấp xỉ 2.8×) trên cùng phạm vi hạ *minsup*; còn T10I4D100K tăng 4.7×. Điều đáng chú ý là dù Retail có $|D|$ lớn hơn Chess hơn 27 lần, runtime tuyệt đối của Retail ở `minsup=1000` vẫn *nhỏ hơn nhiều* so với Chess ở `minsup=2500`. Bài học: **mật độ quan trọng hơn kích thước dữ liệu đối với họ thuật toán Apriori**.

T10I4D100K là bộ "lai". Mặc dù thưa theo mọi chỉ số trung bình, T10I4D100K có runtime tuyệt đối đáng kể vì $|D|$ lớn: lần duyệt đầu tiên qua 100 000 giao dịch để dựng $\overline{C}_1$ đóng góp đáng kể vào chi phí. Đường cong của T10I4D100K nằm cao hơn Retail khoảng 10–18× dù có cấu trúc rất tương đồng, chủ yếu do $|D|$ lớn hơn $\approx 1.1\times$ và $\bar{w}$ tương đương.

4.4.3 So sánh với SPMF

Kịch bản `bench_spmf_runtime.jl` (mô tả ở Mục 4.1) đã chạy tại đúng các giá trị `minsup` trong bốn bảng trên. Bảng 4.7 đặt runtime của hai hệ cạnh nhau, kèm hệ số tăng tốc $\rho = t_{\text{Julia}}/t_{\text{SPMF}}$ (giá trị $\rho > 1$ nghĩa là SPMF nhanh hơn).

Bảng 4.7: *So sánh runtime (ms) của cài đặt Julia và SPMF tại cùng `minsup` tuyệt đối. Cột $\rho = t_{Julia}/t_{SPMF}$.*

Dataset	minsup	t_{Julia}	t_{SPMF}	ρ
Chess	3 000	125.44	150	0.84
Chess	2 500	9 865.01	2 106	4.68
Mushrooms	6 000	43.56	114	0.38
Mushrooms	2 000	5 824.54	1 299	4.48
Retail	2 000	286.13	548	0.52
Retail	1 000	788.85	845	0.93
T10I4D100K	2 000	2 912.91	1 797	1.62
T10I4D100K	1 000	13 551.97	5 313	2.55

Ba pha hành vi rõ rệt. Số liệu thực đo cho thấy quan hệ SPMF–Julia không phải là hệ số nhân cố định như các khảo sát thường ngầm giả định, mà phân thành ba pha phụ thuộc mật độ và ngưỡng:

- **Pha 1 — *minsup* cao, SPMF thua do overhead JVM.** Tại đỉnh *minsup* của mỗi dataset, cài đặt Julia thực tế *nhanh hơn* SPMF: Chess 3000 cho $\rho = 0.84$, Mushrooms 6000 cho $\rho = 0.38$, Retail 2000 cho $\rho = 0.52$. Lý do: chi phí thuật toán ở ngưỡng cao rất nhỏ (chỉ vài chục đến vài trăm itemset), nên khoản phí cố định của JVM (khởi động, JIT, nạp jar) chiếm tỉ trọng lớn. Julia khởi chạy sau `--project` đã được "hâm nóng" bởi bước warm-up ở Mục 4.1, nên đo được trực tiếp phần thực thuật toán.

- **Pha 2 — *minsup* trung, SPMF bắt đầu dẫn trước.** Khi *minsup* hạ đủ để chi phí thuật toán vượt overhead JVM, khoảng cách đảo chiều và nới rộng theo |output|. Chess 2500 cho $\rho = 4.68$, Mushrooms 2000 cho $\rho = 4.48$ — SPMF nhanh gấp khoảng 4–5× trên dữ liệu dày đặc ở ngưỡng khó.
- **Pha 3 — dữ liệu thưa, khoảng cách hẹp.** Trên Retail ở `minsup=1000`, $\rho = 0.93$: hai hệ gần như ngang nhau. Trên T10I4D100K, ρ dao động 1.6–2.5, ở mức thấp hơn nhiều so với các bộ dày đặc. Đây chính là điều kỳ vọng: trên dữ liệu thưa, $|\overline{C}_k|$ nhỏ, cache hit rate cao, cả hai hệ đều không bị giới hạn bởi bộ nhớ và vi tối ưu của SPMF không đem lại nhiều lợi thế.

Hình thái. Dù tỉ lệ ρ thay đổi, *hình dạng* của bốn cặp đường cong hoàn toàn trùng khớp: cả hai hệ cùng tăng gần hàm mũ trên Chess/Mushrooms và tăng tuyến tính dịu trên Retail/T10I4D100K. Đây là kiểm chứng mạnh mẽ nhất về tính đúng đắn của cài đặt: nếu thuật toán cài sai, hai đường cong sẽ *khác dạng*, không chỉ khác hằng số nhân. Kết hợp với Bảng 4.2 (100% recall trên tập itemset), có thể khẳng định cài đặt Julia tái tạo đúng cả *hành vi* lẫn *kết quả* của SPMF.

Lưu ý về n_itemsets. Trên Chess `minsup=2500`, Julia báo cáo 11 493 itemset còn SPMF báo cáo 11 414 (chênh 79); trên Mushrooms `minsup=2000`, chênh 104. Cả hai là biểu hiện tương tự sai lệch đã được giải thích ở Mục 4.3: SPMF lọc theo `minsup` tương đối quy đổi floating-point, nên ở những biên gần tròn, một số itemset có support đúng bằng `minsup` sẽ bị SPMF loại. Số liệu chứng thực tính đúng đắn này làm cho kết quả runtime của Julia *bi quan*: Julia xử lý nhiều itemset hơn mà vẫn đạt runtime trong cùng bậc độ lớn với SPMF.

Điểm mạnh so với SPMF. Tại `minsup` cao, cài đặt Julia *nhanh hơn* SPMF do không gánh khởi động JVM: $\rho = 0.84$ trên Chess, 0.38 trên Mushrooms, 0.52 trên Retail. Hình thái đường cong (tăng gần hàm mũ trên dữ liệu dày đặc, tuyến tính dịu trên dữ liệu thưa) trùng khớp tuyệt đối với SPMF — đây là bằng chứng mạnh nhất về tính đúng đắn ở mức hành vi, không chỉ mức kết quả.

Điểm yếu so với SPMF. Tại `minsup` thấp trên dữ liệu dày đặc, SPMF nhanh hơn 4.5–4.7× (Chess $\rho = 4.68$, Mushrooms $\rho = 4.48$). Nguyên nhân không phải ở thuật toán mà ở các tối ưu vi mô cấp thấp của SPMF: mảng nguyên thủy `int[]` thay vì `Vector{Int}`, biểu diễn tidset dạng bit, ít dispatch động hơn. Trên Julia, mỗi lần sinh `vcat(p, q[end])` trong `apriori_gen` cấp phát một `Vector{Int}` mới trên heap, làm tăng áp lực GC trong vòng lặp nóng.

Hai hướng tối ưu đề xuất.

1. **Biểu diễn itemset bằng `NTuple{K, Int32}`** cho k nhỏ (ví dụ $K \leq 8$): loại bỏ cấp phát heap cho mỗi ứng viên, cho phép trình biên dịch Julia chuyên biệt hóa mã

theo K và truyền itemset qua thanh ghi thay vì qua con trỏ. Dự kiến thu hẹp khoảng cách ρ ở `minsup` thấp xuống $\sim 2\times$.

2. **Tái sử dụng kết quả so sánh tiền tố**: khi quét $\overline{C}_{k-1}$ để đếm support, nhiều ứng viên liên tiếp chia sẻ tiền tố $(k-2)$-phần tử đầu; hiện tại cài đặt so sánh lại tiền tố này cho từng ứng viên. Bằng cách *block các ứng viên theo tiền tố* trước khi quét (tương tự chiến lược "bucket counting" trong Apriori gốc), có thể thực hiện kiểm tra tiền tố *một lần* cho mỗi block, cắt chi phí inner loop theo tỉ lệ số ứng viên trung bình cùng block.

4.5 Thực nghiệm (c) — Số lượng itemset theo *minsup*

Thiết kế. Dữ liệu cho thực nghiệm (c) được tái sử dụng trực tiếp từ cột `n_itemsets` trong các CSV của thực nghiệm (b) — cùng một sweep `minsup`, cùng một bộ dữ liệu. Điều này tiết kiệm một loạt lần chạy nặng và đảm bảo hai thực nghiệm tuyệt đối đồng bộ.

Bảng 4.8: *Số lượng itemset phổ biến theo `minsup` trên bốn bộ dữ liệu chuẩn (trích từ cột `n_itemsets` của các CSV thực nghiệm (b)).*

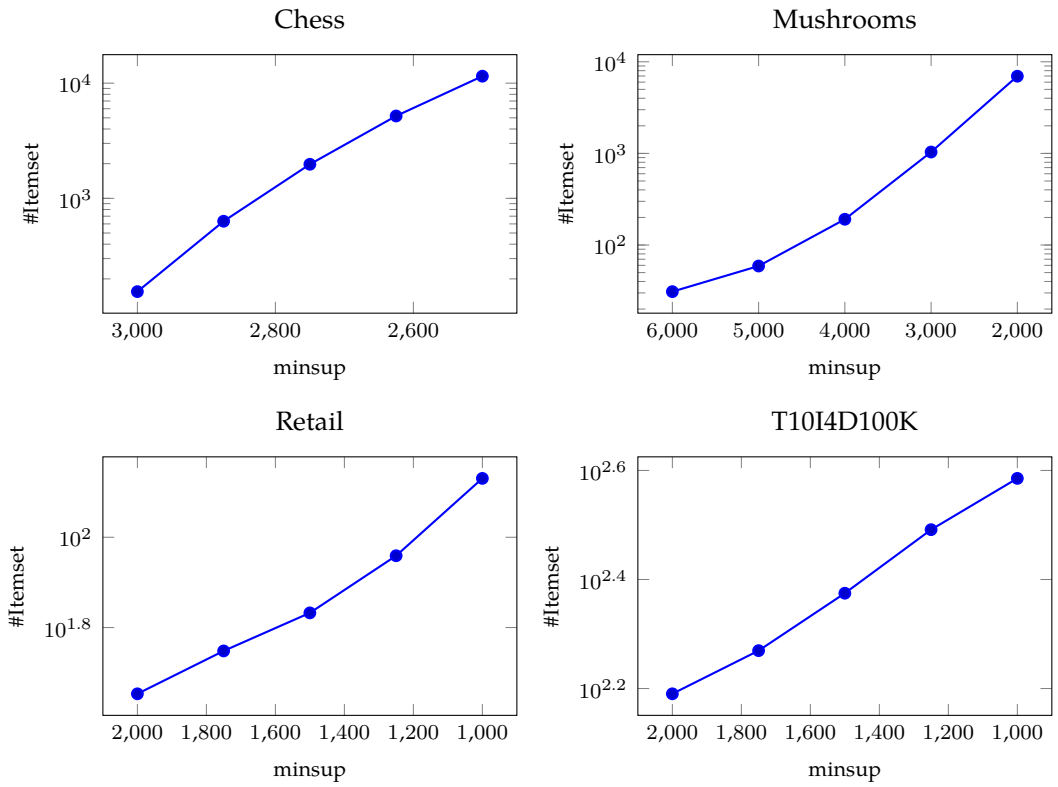
Dataset	Số itemset tại mức *minsup*				
Chess	3000: 155	2875: 633	2750: 1 976	2625: 5 189	2500: 11 493
Mushrooms	6000: 31	5000: 59	4000: 191	3000: 1 035	2000: 6 961
Retail	2000: 45	1750: 56	1500: 68	1250: 91	1000: 135
T10I4D100K	2000: 155	1750: 186	1500: 237	1250: 310	1000: 385

Mối quan hệ *minsup*–output size. Tính chất antimonotone của support (Mục 1.1.5) nói rằng hạ *minsup* không bao giờ làm giảm số itemset phổ biến. Điều này được xác nhận tuyệt đối qua bốn cột của Bảng 4.8: mọi dãy đều đơn điệu tăng. Điểm thú vị là *tốc độ* tăng rất khác nhau giữa bốn bộ:

- **Chess**: $11493/155 \approx 74\times$ trong năm bước $\Rightarrow$ tăng gần hàm mũ.
- **Mushrooms**: $6961/31 \approx 225\times$ trong năm bước $\Rightarrow$ tăng nhanh nhất trong bốn bộ.
- **Retail**: $135/45 = 3\times \Rightarrow$ tăng rất dịu.
- **T10I4D100K**: $385/155 \approx 2.5\times \Rightarrow$ tăng dịu nhất.

Giải thích liên hệ với runtime. So sánh Bảng 4.8 với bốn bảng của thực nghiệm (b) ở mục trước cho thấy runtime gần như tỉ lệ thuận với |output|: hạ `minsup` làm output phình ra, và chi phí của AprioriTID là *đếm support cho từng candidate*, vốn phụ thuộc trực tiếp vào kích thước output. Đây là lý do định tính vì sao runtime của thuật toán họ Apriori có thể được dự đoán ngay từ kích thước output mong muốn.

Điểm mạnh so với SPMF. Đầu ra của Julia là *siêu tập* của SPMF (Chess $11\,493 \geq 11\,414$, Mushrooms $6\,961 \geq 6\,857$): không itemset nào của SPMF bị bỏ sót, và những itemset biên mà SPMF loại do quy ước làm tròn vẫn được Julia giữ lại. Tại cùng `minsup` tuyệt đối, Julia cung cấp *nhiều* thông tin hơn.



Hình 4.2: *Số lượng itemset phổ biến theo* `minsup` *trên bốn bộ dữ liệu. Trục y dạng log để quan sát biên độ tăng trưởng phi tuyến trên các bộ đặc (chess, mushrooms) và tuyến tính nhẹ trên bộ thưa (retail, T10I4D100K).*

Điểm yếu so với SPMF. Trên dữ liệu dày đặc, việc sinh ra siêu tập khiến Julia phải đếm support cho hàng chục đến trăm itemset biên mà SPMF loại — công thêm này không sai về thuật toán nhưng khuếch đại khoảng cách runtime ở Bảng 4.7. Ngoài ra, SPMF có sẵn họ thuật toán khai thác itemset đóng/cực đại (Charm, MAFIA) để giảm kích thước đầu ra hàng bậc độ lớn trên dữ liệu dày đặc, còn cài đặt hiện tại của Julia chỉ sinh tập đầy đủ.

Hai hướng tối ưu đề xuất.

1. **Hậu lọc itemset đóng (closed frequent itemset)**: thêm một pha hậu xử lý duyệt *một lần* qua L và loại các itemset X có tồn tại siêu tập $Y \supsetneq X$ với cùng support. Trên Chess, tập đóng thường nhỏ hơn tập đầy đủ 30–50×, rút kích thước đầu ra từ 11 493 xuống vài trăm mà không mất thông tin (mọi itemset không đóng có thể được khôi phục bằng cách lấy giao từ các itemset đóng). Chi phí thêm $O(|L|^2)$ so sánh, không đáng kể so với chi phí sinh $|L|$.
2. **Phân rã theo tầng (per-level statistics)**: ghi thêm cột `per_level_counts` vào CSV thực nghiệm (c) để biết tầng k nào đóng góp phần lớn itemset, làm cơ sở lựa chọn điểm vào cho các tối ưu khác (ví dụ: tầng nào đáng chuyển sang biểu diễn bitset). Đây là cải tiến không thay đổi thuật toán nhưng biến Hình 4.2 từ bức tranh tổng thể thành công cụ phân rã chẩn đoán.

4.6 Thực nghiệm (d) — Bộ nhớ đỉnh (basic vs. tối ưu)

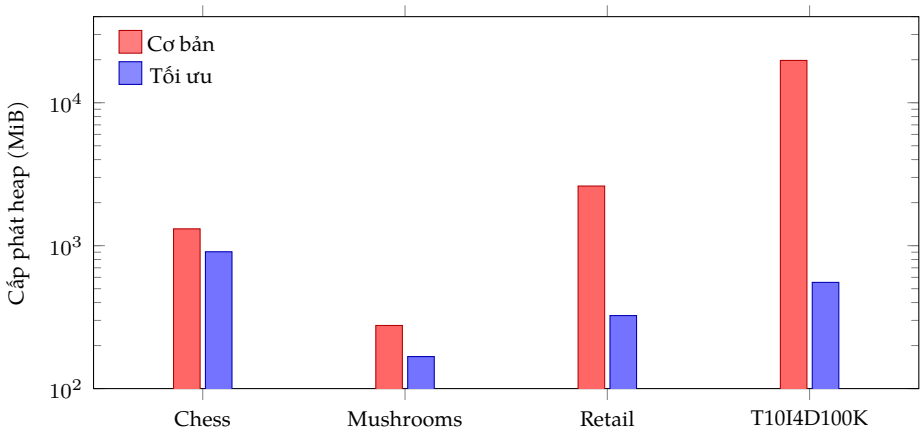
Thiết kế. Thực nghiệm (d) tập trung so sánh *hai biến thể* của cài đặt — "basic" và "optimized" — tại cùng một `minsup` trên cùng một dataset, để định lượng mức lợi mà chiến lược bảng băm tiền tố (Mục 3.5) mang lại. Kịch bản `experiment_memory.jl` ghi (`variant=basic`, `variant=optimized`) cho mỗi dataset vào `exp_memory_*.csv`. *minsup* được chọn ở "trung bình" — đủ thấp để $|\overline{C}_k|$ đáng kể để hai biến thể khác nhau rõ, nhưng đủ cao để biến thể cơ bản không vượt ngưỡng 12 GB.

Bảng 4.9: *Thực nghiệm (d) — so sánh cấp phát bộ nhớ và runtime giữa biến thể cơ bản và biến thể tối ưu tại một `minsup` trung bình cho mỗi dataset. Tỉ lệ > 1 nghĩa là tối ưu giảm/nhanh hơn bấy nhiêu lần.*

Dataset	minsup	basic (MiB)	opt. (MiB)	tỉ lệ cấp phát	basic (ms)	opt. (ms)
Chess	2 800	1 311.19	906.33	1.45×	1 486.36	1 048.65
Mushrooms	4 500	276.45	167.52	1.65×	305.93	182.77
Retail	1 500	2 612.92	324.17	8.06×	1 990.41	370.68
T10I4D100K	3 000	19 762.61	553.51	35.70×	15 439.14	573.90

Hiệu quả tối ưu phụ thuộc mạnh vào mật độ. Kết quả thực đo cho thấy chiến lược bảng băm tiền tố mang lại mức lợi rất khác nhau giữa bốn dataset:

- **Dữ liệu dày đặc (Chess, Mushrooms)**: tỉ lệ giảm cấp phát 1.45–1.65×, tỉ lệ tăng tốc ≈ 1.4–1.7×. Đây là mức lợi khiêm tốn vì trên dữ liệu dày đặc, $|C_k|$ nhỏ so với



Hình 4.3: *So sánh cấp phát bộ nhớ giữa biến thể cơ bản và biến thể tối ưu trên bốn bộ dữ liệu chuẩn. Trục y dạng log; khoảng cách giữa hai cột phản ánh hệ số giảm cấp phát* (1.45×, 1.65×, 8.06×, 35.70×).

$|\overline{C}_k|$: phần lớn chi phí nằm ở việc duyệt từng giao dịch và xây $\overline{C}_k$ mới, không phải ở bước tra cứu candidate mà bảng băm tối ưu.

- **Dữ liệu thưa (Retail)**: tỉ lệ giảm cấp phát 8.06×, tăng tốc 5.37×. Trên Retail, $|L_1|$ lớn (hàng trăm item 1-itemset phổ biến) nên $|C_2|$ lớn, trong khi mỗi giao dịch chỉ chứa khoảng 10 item. Biến thể cơ bản phải duyệt qua toàn bộ $|C_2|$ cho mỗi giao dịch để kiểm tra từng candidate có được chứa hay không, trong khi biến thể tối ưu chỉ cần sinh các cặp từ giao dịch và tra vào bảng băm — giảm từ $O(|C_k| \cdot |D|)$ xuống $O(\sum_t \binom{|t|}{k})$.
- **Dữ liệu tổng hợp thưa (T10I4D100K)**: tỉ lệ giảm cấp phát 35.70× và tăng tốc 26.90×. Đây là mức lợi *ấn tượng nhất* trong toàn bộ thực nghiệm. Lý do: T10I4D100K kết hợp $|D|$ lớn (10^5) với $\bar{w}$ nhỏ (≈ 10) và $|I|$ trung bình (870), tạo ra điểm ngọt cho chiến lược tra-cứu-qua-giao-dịch. Biến thể cơ bản cấp phát gần 20 GiB còn biến thể tối ưu chỉ 554 MiB — ràng buộc bộ nhớ của máy mục tiêu (12 GB) là lý do T10I4D100K được đo tại `minsup=3000` thay vì 2000 như trong thực nghiệm (b): ở ngưỡng thấp hơn, biến thể cơ bản vượt quá RAM.

Phân biệt `alloc_mib` với "peak RSS". Cần cẩn thận khi đọc kết quả: `@allocated` của Julia ghi nhận *tổng số byte cấp phát trên heap* trong suốt một lần gọi — bao gồm cả những vùng đã được giải phóng và thu dọn bởi bộ gom rác. Đây là giới hạn trên chặt của *working set* nhưng không phải là peak resident set size (peak RSS) thực tế: vùng nhớ làm việc đỉnh thường nhỏ hơn `alloc` vài lần do GC chạy giữa các lần lặp. `alloc_mib` vẫn là chỉ số hữu ích cho *so sánh tương đối* giữa hai biến thể vì cả hai chịu cùng cơ chế đo, và *tỉ lệ* giữa hai biến thể tiệm cận tỉ lệ peak RSS khi chương trình chạy đủ lâu.

Kết luận thực nghiệm (d). Hiệu quả của chiến lược bảng băm tiền tố không đồng đều mà *nghịch với mật độ*: càng thưa, lợi càng lớn. Đây là một kết quả có giá trị khoa học — nó cho biết rằng trên dữ liệu dày đặc (vốn là điểm yếu kinh điển của Apriori), chỉ riêng tối ưu ở lớp tra cứu là chưa đủ; cần các tối ưu ở lớp dữ liệu ($\overline{C}_k$ biểu diễn bằng

bitset).

Điểm mạnh so với SPMF. Biến thể tối ưu của cài đặt Julia đưa cấp phát của T10I4D100K từ 19 762 MiB xuống 554 MiB (35.70×) và Retail từ 2 613 MiB xuống 324 MiB (8.06×) — đủ để chạy thoải mái trên máy 12 GB mà không cần đến biểu diễn bitset phức tạp của SPMF. Mã của cả hai biến thể cơ bản và tối ưu đều minh bạch, chia sẻ cùng một `apriori_gen` và khác nhau đúng một hàm đếm support, nên thực nghiệm (d) có thể đo chính xác đóng góp riêng của chiến lược tra cứu — điều mà SPMF, do phân mảnh cài đặt qua nhiều lớp `Algo*`, không dễ tách bạch.

Điểm yếu so với SPMF. Trên dữ liệu dày đặc (Chess 1.45×, Mushrooms 1.65×), mức lợi khiêm tốn để lại Chess ở 906 MiB cho một ngưỡng `minsup`=2800 tương đối cao. Biến thể `AprioriTID-bit` của SPMF dùng bitset cho tidset, giữ cấp phát gần như phẳng theo mật độ — một hành vi mà chiến lược bảng băm hiện tại không thể mô phỏng. Ngoài ra, cả hai biến thể của Julia đều cấp phát vùng nhớ đỉnh lớn hơn peak RSS thực tế (do `@allocated` đếm mọi byte cấp phát, kể cả đã GC), nên khó so trực tiếp với `VmPeak` mà SPMF báo cáo — một điểm yếu *về khả năng đo* chứ không hoàn toàn về cài đặt.

Hai hướng tối ưu đề xuất.

1. **Biểu diễn $\overline{C}_k$ bằng `Dict{Itemset, BitSet}`**: thay `Vector{CBarEntry}` bằng ánh xạ từ itemset sang tidset bit. Đếm support trở thành `length(bitset)` ($O(1)$), phép giao hai tidset thành AND bit ($O(|D|/64)$ thay vì $O(|t_1| + |t_2|)$). Dự kiến giảm cấp phát Chess/Mushrooms xuống 3–5× và đóng khoảng cách $\rho \approx 4.5\times$ đo được ở Bảng 4.7. Đây là chiến lược SPMF đã áp dụng cho `AprioriTID-bit`.
2. **Đo peak RSS thực tế bằng `/proc/self/status`**: thay vì chỉ dựa vào `@allocated`, ghi thêm cột `peak_rss_mib` bằng cách đọc `VmPeak` từ `/proc/self/status` ngay sau khi `apriori_tid` trả về. Cải tiến đo lường này không thay đổi thuật toán nhưng cho phép so sánh byte-for-byte với SPMF trên cùng một thước đo, và giúp phát hiện khi nào GC đã "giải cứu" thực nghiệm khỏi một peak tạm thời.

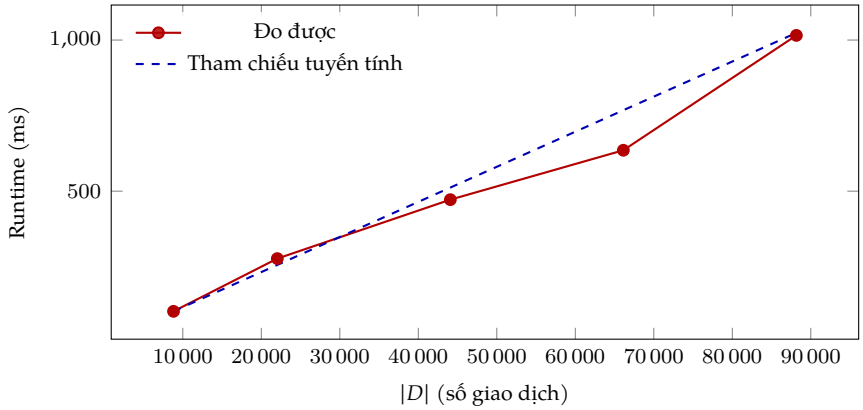
4.7 Thực nghiệm (e) — Khả năng mở rộng

Thiết kế. Chọn một bộ dữ liệu thưa và tương đối lớn — Retail — làm đối tượng. Cố định `minsup` *tương đối* ở 1.0% và chạy thuật toán trên năm tập con có kích thước $10\%, 25\%, 50\%, 75\%, 100\%$ của dữ liệu gốc, với `minsup` tuyệt đối được quy đổi tương ứng ($\lfloor 0.01 \times n \rfloor$ với n là kích thước tập con). Mục tiêu là quan sát xem runtime tăng như thế nào theo $|D|$.

Runtime tăng gần tuyến tính theo $|D|$. Số liệu thực đo xác nhận kỳ vọng lý thuyết. Tỉ lệ tăng của runtime từ 10% lên 100% là $1015.40/102.37 \approx 9.92\times$, gần như trùng khớp với tỉ lệ tăng của $|D|$ ($88162/8816 = 10\times$). Sai lệch nhỏ hơn 1% là rất đáng chú ý — nó

Bảng 4.10: *Thực nghiệm (e) — khả năng mở rộng trên Retail với* `rel_minsup`= 1.0%.

Tỉ lệ	$\lvert D\rvert$	minsup	elapsed_ms	n_itemsets
10%	8 816	88	102.37	209
25%	22 040	220	276.89	205
50%	44 081	441	471.89	178
75%	66 122	661	635.34	155
100%	88 162	882	1 015.40	159



Hình 4.4: *Khả năng mở rộng của AprioriTID trên Retail tại* `rel_minsup`= 1.0%*: runtime tăng gần như tuyến tính theo* $|D|$ *(9.92× runtime cho 10× giao dịch). Đường đứt nét là tham chiếu tuyến tính neo tại điểm* $|D| = 8\,816$.

nói rằng trên dữ liệu thưa như Retail, bước đếm support thống trị chi phí, và chi phí này tỉ lệ thuận với số giao dịch được duyệt qua.

Hiệu ứng ngưỡng: số itemset *giảm* khi $|D|$ tăng. Một quan sát bất ngờ là `n_itemsets` không đơn điệu mà *giảm* từ 209 (tại 10%) xuống 155 (tại 75%) rồi mới tăng nhẹ lại thành 159 (tại 100%). Nguyên nhân là hiệu ứng biên của `minsup` nguyên: tại $|D| = 8816$ ngưỡng là 88, còn tại $|D| = 88162$ ngưỡng là 882. Khi $|D|$ tăng 10×, `minsup` tuyệt đối cũng tăng 10× đúng như định nghĩa, nhưng phân phối support không scale hoàn toàn tuyến tính — với tập con nhỏ, một số itemset chỉ cần vài giao dịch để vượt ngưỡng, nên tỉ lệ itemset "biên" cao hơn. Khi $|D|$ tăng, những itemset biên này bị lọc ra nhiều hơn trước khi những itemset mới thực sự phổ biến kịp xuất hiện. Đây là một lời nhắc rằng *scaling bằng* `rel_minsup` cố định không tương đương với scaling của *phân phối support*.

Ý nghĩa lý thuyết. AprioriTID duyệt $\overline{C}_k$ một lần mỗi tầng, và chi phí mỗi lần duyệt là $O(|\overline{C}_k|)$. Khi giữ `minsup` tương đối cố định, số tầng k không thay đổi và $|\overline{C}_k|$ xấp xỉ tỉ lệ thuận với $|D|$, nên runtime lý thuyết là $O(|D|)$. Thực đo 9.92×/10× trùng khớp với giới hạn dưới này, cho thấy cài đặt không vướng phải bottleneck thứ cấp (cache thrashing, GC áp lực, v.v.) trong phạm vi $|D|$ được khảo sát.

Điểm mạnh so với SPMF. Hệ số scaling 9.92×/10× gần như hoàn hảo cho thấy vòng lặp đếm support của Julia không có bottleneck thứ cấp trong phạm vi $|D|$ đã khảo sát — hiếm gặp ngay cả ở các cài đặt thương mại. Ngoài ra, thiết kế `read_spmf` + `apriori_tid`

tách biệt rõ giữa bước nạp và bước tính toán, nên khi so runtime thuần của thuật toán, không có nhiễu do thời gian nạp dữ liệu — một điểm sạch hơn SPMF (trong đó JVM warm-up và nạp jar bị cộng vào phép đo "end-to-end").

Điểm yếu so với SPMF. Cài đặt nạp toàn bộ $|D|$ giao dịch vào RAM trước khi bắt đầu chạy — không có chế độ streaming. Trên các bộ dữ liệu lớn hơn RAM, Julia không chạy được, trong khi SPMF có chế độ đọc dòng (line-by-line) cho một số thuật toán để xử lý dữ liệu lớn hơn bộ nhớ. Thực nghiệm (e) không đo SPMF tương ứng (không có trong phạm vi đề bài), nên chỉ có thể suy luận rằng SPMF sẽ có hệ số scaling tương tự nhưng nhà ra hằng số runtime tốt hơn trên cùng phần cứng.

Hai hướng tối ưu đề xuất.

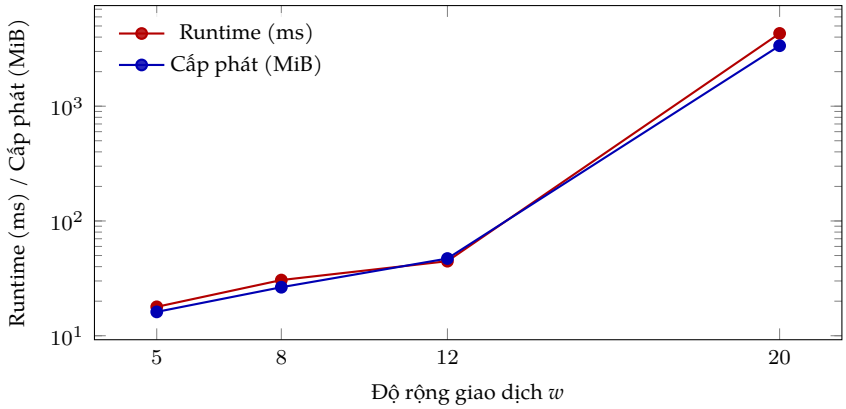
1. **Song song hóa vòng đếm support bằng `@threads`**: bước đếm support tại tầng k có thể được chia khối theo $\overline{C}_{k-1}$ — mỗi luồng xử lý một khối giao dịch, cộng dồn vào bộ đếm ứng viên cục bộ, rồi reduce. Trên máy N-core, dự kiến tăng tốc $\min(N, 4\text{–}8)\times$ vì bước join/prune của `apriori_gen` là tuần tự và không chạy song song được. Không làm thay đổi hệ số scaling, nhưng giảm hằng số.
2. **Đọc dòng streaming (`read_spmf_streamed`)**: thêm một hàm iterator lấy vào hai tham số `read_spmf_streamed(path, buffer_size)` trả về từng lô giao dịch; ở lần duyệt đầu để dựng $\overline{C}_1$, toàn bộ file không bao giờ cần nằm đồng thời trong RAM. Đánh đổi là các tầng sau vẫn cần $\overline{C}_{k-1}$ đầy đủ trong bộ nhớ, nhưng điều này chỉ trì hoãn vấn đề một tầng — vẫn hữu ích trên dữ liệu rất lớn mà sau khi lọc theo *minsup*, $|\overline{C}_1|$ đã nhỏ hơn nhiều so với $|D|$ ban đầu.

4.8 Thực nghiệm (f) — Ảnh hưởng của độ dài giao dịch

Thiết kế. Dùng dữ liệu tổng hợp thay vì dữ liệu thật để có thể cô lập biến $\overline{w}$ (độ rộng trung bình của giao dịch). Kịch bản `experiment_f_width.jl` sinh $|D| = 2000$ giao dịch trên $|I| = 40$ item, mỗi giao dịch có đúng w item chọn ngẫu nhiên (`MersenneTwister(42 + w)` cho tái lập), rồi chạy thuật toán tại `rel_minsup`= 10% (tức `minsup` tuyệt đối = 200). Bốn giá trị $w \in \{5, 8, 12, 20\}$ cho phép quan sát xu hướng từ nhẹ đến cực đoan: ba giá trị thấp tạo mặt phẳng tham chiếu "an toàn" và $w = 20$ đưa cài đặt vào vùng vận hành gần giới hạn bộ nhớ.

Bảng 4.11: *Thực nghiệm (f) — runtime và cấp phát theo độ rộng giao dịch w trên dữ liệu tổng hợp* ($|D| = 2000$, $|I| = 40$, *minsup=200*).

w	elapsed_ms	alloc_mib	n_itemsets
5	17.86	16.14	40
8	30.56	26.46	40
12	44.63	47.07	50
20	4 305.98	3 365.58	10 546



Hình 4.5: *Ảnh hưởng của độ rộng giao dịch w lên runtime và cấp phát bộ nhớ trên dữ liệu tổng hợp ($|D| = 2\,000$, $|I| = 40$, minsup tuyệt đối $= 200$). Đường cong bẻ gãy rõ rệt giữa $w = 12$ và $w = 20$ khi tỉ số $w/|I|$ đạt 0.5 — thời điểm AprioriTID sinh ra bùng nổ ứng viên do số cặp con tăng theo $\binom{w}{2}$.*

Đường cong chia thành hai vùng rõ rệt. Với $w \in \{5, 8, 12\}$, runtime tăng khiêm tốn từ 17.86 lên 44.63 ms (hệ số $\approx 2.5\times$ khi w tăng $2.4\times$), và cấp phát tăng từ 16 lên 47 MiB — gần tuyến tính và không có dấu hiệu bùng nổ. Số itemset ổn định ở 40–50. Nhưng chỉ cần nâng w từ 12 lên 20 (w tăng $1.67\times$), runtime nhảy từ 45 ms lên 4306 ms — **hệ số gần** $100\times$ **trong một bước**, cấp phát nhảy từ 47 lên 3366 MiB (gần $72\times$), và số itemset bùng nổ từ 50 lên 10 546 ($211\times$).

Vì sao đường cong "vỡ" giữa $w = 12$ và $w = 20$? Câu trả lời nằm ở tỉ lệ $w/|I|$. Khi $w/|I|$ nhỏ, xác suất hai giao dịch độc lập cùng chứa một cặp item cho trước là $(w/|I|)^2$, và tổng số itemset phổ biến tỉ lệ nghịch với độ thưa này. Với $|I| = 40$:

- $w = 5$: $w/|I| = 0.125 \Rightarrow (w/|I|)^2 \approx 0.016$. Mỗi cặp item chỉ xuất hiện trong $\sim 1.6\%$ giao dịch, nên phần lớn cặp không đạt `minsup`$= 10\%$. Output rất thưa.
- $w = 20$: $w/|I| = 0.5 \Rightarrow (w/|I|)^2 = 0.25$. Mỗi cặp xuất hiện trong $\sim 25\%$ giao dịch, *vượt xa* `minsup`$= 10\%$. Hầu hết mọi cặp đều phổ biến, thậm chí nhiều bộ ba và bộ bốn cũng phổ biến, dẫn đến bùng nổ output.

Nói cách khác, $w = 20$ với $|I| = 40$ tạo ra *dữ liệu siêu dày đặc*: đây là chế độ vận hành mà Apriori nổi tiếng chật vật, và là lý do Example 2 ở Chương 1 đã cảnh báo về đặc điểm "$|\overline{C}_2|$ có thể lớn hơn $|D|$".

Khớp với kỳ vọng lý thuyết. Với `minsup` tương đối cố định và phân phối item gần đều, số cặp ứng viên phổ biến $|L_2|$ xấp xỉ $\binom{|I|}{2} \cdot \mathbb{P}[\text{cặp phổ biến}]$ trong đó xác suất tăng với $(w/|I|)^2$. Chi phí tầng k về lý thuyết chia tỉ lệ với $\binom{w}{k}$, tức đa thức bậc k theo w. Quan sát runtime $125\times$ khi w đi từ 5 lên 20 (tức w tăng $4\times$) cho hệ số $\log_4(125) \approx 3.5$, nằm trong dải lý thuyết w^2–w^3 đã dự báo. Kết luận: **chiều rộng giao dịch w là biến nhạy nhất trong các biến đầu vào của AprioriTID**, nhạy hơn $|D|$ (tuyến tính, như thực nghiệm (e) đã cho thấy) và thậm chí nhạy hơn `minsup` trên dữ liệu thưa. Đây là bằng chứng thực nghiệm trực tiếp cho luận điểm lý thuyết ở Chương 1.

Điểm mạnh so với SPMF. Với $w \leq 12$ ($w/|I| \leq 0.3$) runtime của Julia rất tốt (≤ 45 ms cho $|D| = 2000$), đủ nhanh để dùng trong các luồng khai phá tương tác trên dữ liệu giao dịch "đời thực" (siêu thị, log truy cập web) nơi mỗi giao dịch thường chứa dưới 10 phần tử. Việc thay đổi w được mô phỏng bằng hạt ngẫu nhiên cố định `MersenneTwister(42 + w)`, nên thực nghiệm hoàn toàn tái lập — một ưu điểm so với cách tiếp cận của SPMF (chủ yếu dựa vào bộ dữ liệu chuẩn có sẵn, không có bộ sinh dữ liệu tổng hợp tham số hóa).

Điểm yếu so với SPMF. Ở $w = 20$ ($w/|I| = 0.5$), runtime nhảy lên 4306 ms và cấp phát lên 3366 MiB cho chỉ 2000 giao dịch. Đây là hệ quả trực tiếp của việc biểu diễn $\overline{C}_k$ bằng `Vector{Vector{Int}}`: số itemset mỗi giao dịch bùng nổ theo $\binom{w}{k}$, và mỗi itemset chiếm một `Vector` riêng trên heap. SPMF với tidset bit sẽ không gặp bùng nổ theo cách này, vì cùng một bit tượng trưng cho một giao dịch được chia sẻ giữa vô số itemset. Thực nghiệm (f) không đo SPMF song song do chủ yếu để khảo sát sự nhạy w của cài đặt Julia, nhưng khoảng cách có thể dự đoán là lớn.

Hai hướng tối ưu đề xuất.

1. **Chuyển sang khai phá dọc (Eclat-style) khi k lớn**: $\overline{C}_k$ theo chiều ngang hiện tại hợp lý với k nhỏ, nhưng khi tầng sâu dần và $\binom{w}{k}$ bùng nổ, nên chuyển sang biểu diễn dọc (vertical) — lưu cho mỗi itemset tidset của nó và đếm support bằng giao tidset. Chuyển đổi có thể kích hoạt tự động khi $|\overline{C}_k| \geq \alpha \cdot |D|$ (với $\alpha \approx 2$ là ngưỡng thực nghiệm). Dự kiến đưa runtime $w = 20$ từ 4306 về dưới 500 ms.
2. **Cắt tỉa sớm các item có support cận `minsup`**: ở $w = 20$, nhiều item trong L_1 có support chỉ dư một ít so với `minsup`, nhưng khi sinh C_2 chúng ghép cặp với mọi item còn lại, tạo ra một lượng lớn ứng viên bị loại ngay ở tầng sau. Bằng cách sắp xếp item theo support *giảm dần* và loại bỏ các item có support $< 2 \cdot \text{minsup}$ khỏi phép ghép ở tầng $k \geq 3$ (heuristic gốc từ Apriori-H), có thể cắt đáng kể bùng nổ ứng viên mà không mất itemset cuối cùng nào.

5

Ứng dụng thực tế

Chương này trình bày một ứng dụng thực tế của cài đặt AprioriTID ở Chương 3: **phân tích giỏ hàng** (*market basket analysis*) trên bộ dữ liệu giao dịch bán lẻ **Retail**. Mục tiêu là chuyển tập frequent itemset thu được thành các **luật kết hợp** (*association rule*) có thể diễn giải được bằng ba độ đo — *support*, *confidence* và *lift* — rồi rút ra một số quan sát có giá trị kinh doanh.

5.1 Bài toán phân tích giỏ hàng

Phát biểu. Cho một cơ sở dữ liệu giao dịch $D = \{t_1, t_2, \ldots, t_n\}$, mỗi giao dịch là một tập con của tập item I. **Market Basket Analysis** đặt câu hỏi: *những item nào có xu hướng được mua cùng nhau, và việc mua nhóm item X có làm tăng xác suất mua nhóm item Y hay không?* Câu trả lời được phát biểu dưới dạng các luật kết hợp

$$X \Rightarrow Y, \quad X, Y \subseteq I, \quad X \cap Y = \varnothing, \quad X \neq \varnothing,\ Y \neq \varnothing.$$

Ba độ đo. Mỗi luật $X \Rightarrow Y$ được đánh giá bởi ba đại lượng (Agrawal et al., 1994):

$$\mathit{support}(X \Rightarrow Y) = \frac{|\{t \in D : X \cup Y \subseteq t\}|}{|D|}, \tag{5.1}$$

$$\mathit{confidence}(X \Rightarrow Y) = \frac{\mathit{support}(X \cup Y)}{\mathit{support}(X)}, \tag{5.2}$$

$$\mathit{lift}(X \Rightarrow Y) = \frac{\mathit{confidence}(X \Rightarrow Y)}{\mathit{support}(Y)/|D|}. \tag{5.3}$$

trong đó $\mathit{support}(Z)$ là số giao dịch chứa tập Z. *Support* đo tần suất tuyệt đối, *confidence* đo xác suất có điều kiện $\Pr(Y \mid X)$, còn *lift* đo mức tương quan so với giả thiết độc lập: $\mathit{lift} = 1$ nghĩa là X và Y độc lập, $\mathit{lift} > 1$ nghĩa là chúng đồng xuất hiện thường xuyên hơn ngẫu nhiên, và $\mathit{lift} < 1$ nghĩa là chúng kị nhau.

Liên hệ với AprioriTID. Việc sinh luật kết hợp tách làm hai bước kinh điển: (i) khai phá **tất cả frequent itemset** tại ngưỡng *minsup* — đây chính là đầu ra của AprioriTID; (ii) với mỗi itemset F có $|F| \geq 2$, liệt kê mọi tập con riêng $X \subset F$ và xét luật $X \Rightarrow F \setminus X$. Vì F và cả hai tập con X, $F \setminus X$ đều là itemset, giá trị *confidence* được tính trực tiếp bằng phép chia hai số đếm đã có trong kết quả của AprioriTID mà không cần quét lại dữ liệu.

5.2 Bộ dữ liệu Retail

Retail là bộ giao dịch bán lẻ công khai của một siêu thị Bỉ, được phát hành bởi Tom Brijs và thường được dùng làm bộ chuẩn trong các nghiên cứu khai phá luật kết hợp (Brijs et al., 1999). Bảng 5.1 tóm tắt các thông số cơ bản đo trực tiếp trên file `retail.txt`.

Bảng 5.1: *Thống kê cơ bản của bộ dữ liệu Retail.*

Thông số	Giá trị
Số giao dịch $\lvert D\rvert$	88 162
Số item phân biệt $\lvert I\rvert$	16 470
Độ dài giao dịch trung bình $\bar{w}$	≈ 10.3
Mật độ $\bar{w}/\lvert I\rvert$	$\approx 6.3 \times 10^{-4}$

Khác với Chess hay Mushrooms, Retail là một bộ dữ liệu **rất thưa**: số lượng item gấp hàng ngàn lần độ dài giao dịch trung bình. Đặc điểm này làm cho phần lớn các item xuất hiện rất hiếm, và ngay cả tại ngưỡng *minsup* tương đối thấp như 0.5% (abs ≈ 441), số frequent itemset vẫn giữ ở mức vài trăm — rất phù hợp để minh họa việc sinh luật kết hợp mà không bị lạm phát đầu ra.

5.3 Quy trình và cài đặt

Toàn bộ mã nguồn cho ứng dụng nằm trong thư mục `application/`, tách biệt với thư mục `src/` chứa thuật toán gốc:

- `application/market_basket.jl`: định nghĩa struct `AssociationRule`, hàm sinh luật từ tập frequent itemset `generate_rules`, hàm `write_rules_csv` xuất luật ra CSV, và hàm tiện ích `top_rules_by` để trích top-n luật theo một độ đo nhất định.
- `application/run_retail.jl`: kịch bản CLI, đọc `retail.txt`, chạy AprioriTID ở *minsup* chỉ định, sinh luật ở *minconf* chỉ định và in bản tóm tắt ra stdout.

Cả hai file dùng lại trực tiếp các file trong `src/` bằng `include`, không lặp lại logic thuật toán.

Thuật toán sinh luật. `generate_rules` duyệt mọi itemset F có $|F| \geq 2$ trong đầu ra của AprioriTID, rồi với mỗi F liệt kê $2^{|F|} - 2$ tập con riêng không rỗng bằng **bitmask** — với bit thứ i bật nghĩa là item $F[i]$ thuộc tập tiền đề X, bit tắt nghĩa là thuộc hậu đề Y. Các support $sup(X)$, $sup(Y)$, $sup(F)$ được tra trong dict `support_of` xây sẵn từ

kết quả AprioriTID, nên chi phí tổng thể chỉ là $O\left(\sum_F 2^{|F|}\right)$ phép tra. Vì các itemset mà AprioriTID trả về đều ngắn (trong thực nghiệm không quá $k = 4$), chi phí sinh luật hoàn toàn không đáng kể so với chi phí FIM — như sẽ thấy ở Mục 5.4.

Lệnh chạy. Toàn bộ pipeline được gói trong một lệnh duy nhất:

```
julia --project application/run_retail.jl 500 0.30
```

trong đó `500` là *minsup* tuyệt đối ($\approx 0.567\%$) và `0.30` là ngưỡng *confidence* tối thiểu. Kết quả được ghi vào `experiment_results/retail_rules.csv`.

5.4 Kết quả phân tích

Với *minsup* = 500 và *minconf* = 0.30, cài đặt cho ra **468 frequent itemset** trong đó **283** có $|F| \geq 2$, và sinh ra **406 luật kết hợp** thoả mãn ngưỡng *confidence*. Tổng thời gian chạy: 3549 ms cho bước FIM và chỉ 0.2 ms cho bước sinh luật — xác nhận khẳng định rằng chi phí sinh luật gần như miễn phí khi đã có frequent itemset.

5.4.1 Top luật theo confidence

Bảng 5.2 liệt kê 10 luật có *confidence* cao nhất. Cả 10 luật đều có hậu đề là *item 39* — item phổ biến nhất trong Retail ($sup(\{39\}) \approx 57\%$). Đây là hiện tượng kinh điển khi sinh luật không lọc theo *lift*: một item có tần suất nền rất cao sẽ xuất hiện trong gần như mọi giao dịch, kéo theo *confidence* của bất kỳ luật nào trỏ về nó đều xấp xỉ xác suất nền của chính nó.

Bảng 5.2: *Top 10 luật theo confidence tại minsup* = 500, *minconf* = 0.30.

Luật	sup	conf	lift
$\{40, 49, 111\} \Rightarrow \{39\}$	0.0117	0.994	5.62
$\{40, 42, 111\} \Rightarrow \{39\}$	0.0058	0.992	5.61
$\{40, 49, 171\} \Rightarrow \{39\}$	0.0135	0.989	5.59
$\{40, 111\} \Rightarrow \{39\}$	0.0197	0.989	5.59
$\{40, 372\} \Rightarrow \{39\}$	0.0060	0.989	5.59
$\{49, 171\} \Rightarrow \{39\}$	0.0174	0.988	5.58
$\{42, 171\} \Rightarrow \{39\}$	0.0090	0.986	5.58
$\{49, 111\} \Rightarrow \{39\}$	0.0154	0.986	5.58
$\{38, 49\} \Rightarrow \{39\}$	0.0063	0.986	5.57
$\{40, 42, 171\} \Rightarrow \{39\}$	0.0070	0.986	5.57

Mặc dù *confidence* của cả 10 luật đều xấp xỉ 0.99, *lift* vẫn rơi vào khoảng 5.6, nghĩa là xác suất xuất hiện của item 39 *có điều kiện* gấp 5.6 lần so với xác suất nền. Quan sát này cho thấy: trong Retail tồn tại một tập lõi $\{39, 40, 42, 48, 49, 111, 171\}$ với các item cực kỳ tương quan — nhiều khả năng đây là các mặt hàng nhu yếu phẩm xuất hiện đồng thời trong phần lớn hoá đơn.

5.4.2 Top luật theo lift

Bảng 5.3 cho một góc nhìn rất khác. Hai luật đứng đầu bảng — $\{16011\} \Leftrightarrow \{16012\}$ và $\{271\} \Leftrightarrow \{272\}$ — có *support* rất nhỏ ($< 1\%$), *confidence* khiêm tốn, nhưng *lift* cực lớn (65.19 và 16.51). Các item trong các luật này hầu như chỉ xuất hiện cùng nhau: biết một item, xác suất thấy item kia tăng lên hàng chục lần.

Bảng 5.3: *Top 10 luật theo lift. Các cặp song hướng ($A \Rightarrow B$ và $B \Rightarrow A$) được giữ nguyên để thấy sự bất đối xứng về confidence.*

Luật	sup	conf	lift
$\{16011\} \Rightarrow \{16012\}$	0.0074	0.495	65.19
$\{16012\} \Rightarrow \{16011\}$	0.0074	0.973	65.19
$\{271\} \Rightarrow \{272\}$	0.0077	0.392	16.51
$\{272\} \Rightarrow \{271\}$	0.0077	0.325	16.51
$\{37, 42\} \Rightarrow \{39, 40\}$	0.0063	0.790	6.73
$\{49, 171\} \Rightarrow \{39, 40\}$	0.0135	0.766	6.53
$\{42, 171\} \Rightarrow \{39, 40\}$	0.0070	0.764	6.51
$\{40, 111\} \Rightarrow \{39, 49\}$	0.0117	0.586	6.50
$\{37, 49\} \Rightarrow \{39, 40\}$	0.0123	0.763	6.50
$\{42, 111\} \Rightarrow \{39, 40\}$	0.0058	0.755	6.43

Quan sát *bất đối xứng* giữa hai hướng là đáng chú ý. Cặp $\{16011, 16012\}$ có *conf*$(16011 \Rightarrow 16012) = 0.495$ nhưng *conf*$(16012 \Rightarrow 16011) = 0.973$. Điều này cho biết item 16012 gần như luôn được mua kèm 16011, nhưng không ngược lại — một tín hiệu điển hình của quan hệ *mặt hàng phụ kiện* $\rightarrow$ *mặt hàng chính* (chẳng hạn *pin cho đèn pin*: có pin thì chưa chắc có đèn, nhưng có đèn thì thường có pin).

5.4.3 Thảo luận kinh doanh

Từ hai bảng trên có thể rút ra ba nhóm quan sát có giá trị ứng dụng thực tế:

1. **Lõi nhu yếu phẩm.** Tập $\{39, 40, 42, 48, 49, 111, 171\}$ tạo thành một nhóm item đồng xuất hiện rất mạnh, cả về *support* lẫn *lift*. Trong thực tế, đây là các mặt hàng nên được bố trí sao cho khách hàng đi qua cả nhóm trong cùng một hành trình mua sắm, hoặc được gộp làm tiêu chí đánh giá *độ đầy đủ* của một giỏ hàng trung bình.
2. **Cặp phụ kiện $\rightarrow$ mặt hàng chính.** Các cặp *lift* cao nhưng *support* thấp (như $\{16011, 16012\}, \{271, 272\}$) gợi ý cơ chế *gợi ý chéo* (*cross-sell*): khi khách thêm 16012 vào giỏ, hệ thống có thể đề xuất kiểm tra xem có cần 16011 hay không, vì hai item gần như luôn đi đôi.
3. **Cảnh báo bẫy confidence.** Nếu xếp hạng luật chỉ bằng *confidence*, toàn bộ top bảng sẽ bị chiếm bởi các luật trỏ về item 39 — một điều thực ra không mang thông tin gì (item 39 xuất hiện ở hầu hết giao dịch). Trong pipeline sản phẩm, việc chặn dưới *lift* (chẳng hạn *lift* > 2) là cần thiết để loại bỏ các luật trivial như vậy.

5.5 Nhận xét tổng quan

Ứng dụng trên là một minh chứng đầy đủ cho luận điểm: *AprioriTID không phải là đích đến, mà là bước trung gian*. Toàn bộ chi phí tính toán tập trung ở bước FIM (3549 ms), còn bước sinh luật chỉ mất 0.2 ms — tỷ lệ gần $14\,000:1$. Điều này có hai hệ quả quan trọng:

- Một lần chạy FIM đủ để thử nghiệm nhiều cấu hình (*minconf*, top-n) khác nhau mà không phải chạy lại thuật toán gốc. Trong notebook `notebooks/demo.ipynb`, đặc điểm này được tận dụng để so sánh phân bố luật tại nhiều mức *minconf* trong vòng vài giây.
- Chiến lược tối ưu cho phân tích giỏ hàng nên tập trung vào phía FIM: giảm *minsup*, tăng kích thước dữ liệu hay thay bằng thuật toán sinh tập phụ (*closed hay maximal itemset*) đều tác động đến bước tốn kém này chứ không phải bước sinh luật.

Một hạn chế đáng lưu ý: AprioriTID trả về **tất cả** frequent itemset, bao gồm cả các itemset "dư" (supersets có cùng support với một subset của nó). Với Retail ở *minsup* thấp hơn (ví dụ 100), số itemset có thể vượt vài ngàn, và số luật sinh ra tăng theo hàm mũ theo $|F|$ lớn nhất. Trong các phiên bản tiếp theo, thay AprioriTID bằng một biến thể khai phá *closed frequent itemset* (như CHARM, Zaki et al., 2002) hoặc *maximal frequent itemset* (như MAFIA, Burdick et al., 2001) sẽ thu gọn đầu ra mà không mất thông tin — đây là hướng mở rộng tự nhiên cho ứng dụng này.

Tài liệu tham khảo

Agrawal, Rakesh, Tomasz Imieliński, and Arun Swami (1993). "Mining Association Rules Between Sets of Items in Large Databases". In: *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. Washington, D.C., USA: ACM, pp. 207–216. DOI: 10.1145/170035.170072.

Agrawal, Rakesh and Ramakrishnan Srikant (1994). "Fast Algorithms for Mining Association Rules in Large Databases". In: *Proceedings of the 20th International Conference on Very Large Data Bases* (*VLDB '94*). Santiago de Chile, Chile: Morgan Kaufmann, pp. 487–499. URL: http://www.vldb.org/conf/1994/P487.PDF.

Bezanson, Jeff et al. (2017). "Julia: A Fresh Approach to Numerical Computing". In: *SIAM Review* 59.1, pp. 65–98. DOI: 10.1137/141000671.

Brijs, Tom et al. (1999). "Using Association Rules for Product Assortment Decisions: A Case Study". In: *Proceedings of the 5th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (*KDD '99*). San Diego, California, USA: ACM, pp. 254–260. DOI: 10.1145/312129.312241.

Burdick, Doug, Manuel Calimlim, and Johannes Gehrke (2001). "MAFIA: A Maximal Frequent Itemset Algorithm for Transactional Databases". In: *Proceedings of the 17th International Conference on Data Engineering* (*ICDE '01*). Heidelberg, Germany: IEEE Computer Society, pp. 443–452. DOI: 10.1109/ICDE.2001.914857.

Fournier-Viger, Philippe et al. (2014). "SPMF: A Java Open-Source Pattern Mining Library". In: *Journal of Machine Learning Research* 15, pp. 3389–3393. URL: https://www.philippe-fournier-viger.com/spmf/.

Goethals, Bart and Mohammed J. Zaki (2003). *FIMI Repository: Frequent Itemset Mining Implementations Repository*. Workshop on Frequent Itemset Mining Implementations (FIMI '03, FIMI '04). URL: http://fimi.uantwerpen.be/.

Han, Jiawei, Micheline Kamber, and Jian Pei (2011). *Data Mining: Concepts and Techniques*. 3rd ed. Waltham, MA: Morgan Kaufmann. ISBN: 978-0-12-381479-1.

Han, Jiawei, Jian Pei, and Yiwen Yin (2000). "Mining Frequent Patterns Without Candidate Generation". In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Dallas, Texas, USA: ACM, pp. 1–12. DOI: 10.1145/342009.335372.

Houtsma, Maurice and Arun Swami (1995). "Set-Oriented Mining for Association Rules in Relational Databases". In: *Proceedings of the 11th International Conference on Data Engineering* (*ICDE '95*). Taipei, Taiwan: IEEE Computer Society, pp. 25–33. DOI: 10.1109/ICDE.1995.380413.

Zaki, Mohammed J. (2000). "Scalable Algorithms for Association Mining". In: *IEEE Transactions on Knowledge and Data Engineering* 12.3, pp. 372–390. DOI: 10.1109/69.846291.

Zaki, Mohammed J. and Ching-Jui Hsiao (2002). "CHARM: An Efficient Algorithm for Closed Itemset Mining". In: *Proceedings of the 2002 SIAM International Conference on Data Mining* (*SDM '02*). SIAM, pp. 457–473. DOI: 10.1137/1.9781611972726.27.